```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.CrazyCampus.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S08_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot scene host: register mock services -&gt; run seven steps -&gt; switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
            // S1 campus stage mounts itself in code (G08-T005); S2 scene authoring replaces this.
            var host = new GameObject("CampusStageView");
            host.AddComponent&lt;CampusStageView&gt;();
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
```

```
namespace Sy.CrazyCampus.Boot
{
    /// &lt;summary&gt;
    /// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
    /// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
    /// &lt;/summary&gt;
    public sealed class BootSequence
    {
        public const int TotalSteps = 7;
        readonly Func&lt;bool&gt; _switchScene;
        readonly List&lt;string&gt; _lines = new();
        public BootSequence(Func&lt;bool&gt; switchScene)
        {
            _switchScene = switchScene;
        }
        public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
        public bool RunAll()
        {
            bool ok = true;
            ok &amp;= Run(1, "Logger", LoggerStep);
            ok &amp;= Run(2, "Time", TimeStep);
            ok &amp;= Run(3, "Config", ConfigStep);
            ok &amp;= Run(4, "Save", SaveStep);
            ok &amp;= Run(5, "ILocale", LocaleStep);
            ok &amp;= Run(6, "Ads.Preload", AdsStep);
            ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
            return ok;
        }
        bool Run(int n, string name, Func&lt;bool&gt; step)
        {
            bool result;
            try
            {
                result = step();
            }
            catch (Exception e)
            {
                Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
                result = false;
            }
            string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
            _lines.Add(line);
            Logger.Info("BOOT", line);
            return result;
        }
        static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
        static bool TimeStep()
        {
            var time = ServiceLocator.Get&lt;ITimeService&gt;();
            return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
```

```
        }
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock = no hot update
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.CrazyCampus.Boot
{
    /// &lt;summary&gt;
    /// Dev-time mock service set (SY_MOCK scope). All seven boot steps depend on this set.
    /// T009: the ads service rides the channel suite (PlatformBootstrap) so
    /// ShowRewarded consumption is category-capped on the shared AdCap in
    /// every channel - the offline doubler pre-check and the consume can
    /// never disagree.
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static readonly AdCapPolicy AdCap = new AdCapPolicy();
        /// &lt;summary&gt;Resolved channel suite (T009). Non-null once RegisterAll
        /// ran; the suite's ads service is what ServiceLocator hands out.&lt;/summary&gt;
        public static PlatformSuite Suite { get; private set; }
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; // idempotent: double register would throw
```

```
        ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
        ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
        // G08-T012: the campus rides the shared Sy.Core SaveService (version
        // migration + timestamp faces inherited, zero new save service built).
        ServiceLocator.Register&lt;ISaveService&gt;(new SaveService());
        ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
        // T009: channel suite wiring. PlatformBootstrap gates SY_WECHAT /
        // SY_DOUYIN builds to the real channel suites; editor/batchmode stays
        // mock. Ads consume from the shared AdCap in every channel so the
        // offline-doubler pre-check/consume same-instance law survives the
        // suite switch.
        Suite = PlatformBootstrap.Create(AdCap);
        ServiceLocator.Register&lt;PlatformSuite&gt;(Suite);
        ServiceLocator.Register&lt;IAdsService&gt;(Suite.Ads);
        Logger.Info("BOOT", "services registered (suite=" + PlatformBootstrap.SuiteIdFor(Suite.Channel) + ")");
    }
}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK: device time doubles as server time until the real chain lands.&lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;No cloud: local storage is the authority (zero-server doctrine).&lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
```

```
        /// &lt;summary&gt;Missing key falls back to the key itself with a warning.&lt;/summary&gt;
        public string GetText(string key, params string[] args)
        {
            Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
            return key;
        }
        // U184 real-device CJK face (B569 chain step 3): LegacyRuntime.ttf ships
        // no zh glyphs, so the 84 zh-CN display lines render blank on real
        // devices. The B569 fleet-cast NotoSansSC subset (201KB, cmap-verified
        // missing=0 over the G08 452-char census) rides Resources as
        // NotoSansSC-CrazySubset.otf; the view resolves this name (see
        // CampusStageView.UiFont) and degrades to LegacyRuntime on a miss.
        public FontSet GetFontSet()
            =&gt; new FontSet("NotoSansSC-CrazySubset.otf", "NotoSansSC-CrazySubset.otf", "NotoSansSC-CrazySubset.otf");
    }
}
// CampusAbout.cs
// CampusAbout.cs - the About-panel disclosure composer (G08-T036).
// The AIGC-label duty face (FE-20260924-B-11): 17 TJ AI-assisted art
// assets landed in the catalog (B516), so the About panel carries the
// explicit AI-content mark, the zero-RNG ten-pull disclosure and the
// daily ad-cap line. Every number composes live off the frozen tables -
// the panel and the gameplay read the same objects, zero hardcoded panel
// copies (the P06 about law). The aigc line ships the ART-ONLY wording
// today; the art+audio upgrade is a one-line flip through the texts
// pair - the two versions never coexist (the G11 T017 law).
using Sy.Platform;
namespace Sy.CrazyCampus
{
    public static class CampusAbout
    {
        /// &lt;summary&gt;Panel chrome: the bottom-bar button label.&lt;/summary&gt;
        public static string ButtonLabel()
            =&gt; CampusTexts.Text(CampusTexts.KeyAboutButton);
        /// &lt;summary&gt;Panel chrome: the frame title.&lt;/summary&gt;
        public static string Title()
            =&gt; CampusTexts.Text(CampusTexts.KeyAboutTitle);
        /// &lt;summary&gt;The explicit AI-content mark (current = art-only
        /// wording; the audio flip lands with the dim7 audio batch).&lt;/summary&gt;
        public static string AigcLine()
            =&gt; CampusTexts.Text(CampusTexts.KeyAboutAigcLine);
        /// &lt;summary&gt;The zero-RNG ten-pull disclosure - pool / outfit /
        /// title counts read the live costume table, never a copy.&lt;/summary&gt;
        public static string OddsLine()
            =&gt; string.Format(CampusTexts.Text(CampusTexts.KeyAboutOddsLine),
                CampusCostumeTable.PoolCount,
                CampusCostumeTable.OutfitCount,
                CampusCostumeTable.TitleCount);
        /// &lt;summary&gt;The daily ad-cap disclosure - the four category caps
        /// read AdCapPolicy live (rescue / double / speedup / direct).&lt;/summary&gt;
```

```
        public static string AdCapLine()
            =&gt; string.Format(CampusTexts.Text(CampusTexts.KeyAboutAdcapLine),
                AdCapPolicy.CategoryCap(AdCategories.Rescue),
                AdCapPolicy.CategoryCap(AdCategories.Double),
                AdCapPolicy.CategoryCap(AdCategories.Speedup),
                AdCapPolicy.CategoryCap(AdCategories.Direct));
        /// &lt;summary&gt;The age-rating line (SP08 verdict: 8+).&lt;/summary&gt;
        public static string AgeLine()
            =&gt; CampusTexts.Text(CampusTexts.KeyAboutAgeLine);
        /// &lt;summary&gt;Panel chrome: the dismiss hint.&lt;/summary&gt;
        public static string CloseLabel()
            =&gt; CampusTexts.Text(CampusTexts.KeyAboutClose);
    }
}
// CampusAffinity.cs
// CampusAffinity.cs - the youth affinity line data face (G08-T013 slice A).
// SYS06: the light campus-romance face the user ordered 2026-09-20 ("a light
// romance face is fine, nothing over the line"). Guardrails live first and
// are frozen here as code (design doc sect7): stage-gated unlock (guardrail
// 1 - kindergarten and primary carry zero romance beats, pure friendship
// only), note/text/emote/pixel carriers only (guardrail 2 - zero physical
// contact), teacher roles never enter the pool (guardrail 3). The face is
// derive-on-read over the event engine log (the CampusCodex law: zero new
// game state, zero RNG, zero wall-clock): the T006 booking ledger (E04
// truce-chain tail +1, E09 umbrella card +1) formalizes into per-NPC values
// through the codex seat mapping, and the stage-3/4 growth beats ride the
// frozen 30-school-day cadence. The graduation-season confession resolves
// the dual comedy endings (confessed / friend card) and hands the album
// photo token to the caller - slice B wires Album.AddPhoto and the beat
// emission. Zero server faces (red line 12).
using System.Collections.Generic;
namespace Sy.CrazyCampus
{
    public static class CampusAffinity
    {
        // frozen constants (doc source: G08 numeric pack sect10 affinity face)
        public const int RomanceUnlockStage = 3;    // guardrail 1: kindergarten/primary = zero romance beats
        public const int GrowthCadenceDays = 30;    // one growth beat per 30 school days inside stages 3/4
        public const int GrowthBeatsPerStage = 3;   // the 90-day years cap the beats at three per stage
        public const int ConfessionThreshold = 8;   // growth 6 + both deskmate arcs (truce + umbrella) = 8
        public const string TargetNpc = "deskmate"; // the stage-3/4 growth line is the deskmate arc (sect7)
        // guardrail 3: the pool rides the cast seats (design doc 8.1) -
        // teacher stems never enter, the line stays student-to-student.
        public static readonly string[] NpcPool = { "deskmate", "artist", "athlete" };
        public static readonly string[] TeacherStems = { "head", "math", "pe", "music" };
        // guardrail 2: the carrier tokens - notes, words, emotes and pixel
        // staging only. Middle school = the deskmate friendship sprouts
        // (borrow a note, pass the water bottle, duty together - no
        // confession); high school = the shy goodwill beats.
        public static readonly string[] MiddleTokens = { "borrow_note", "pass_water", "duty_together" };
```

```
public static readonly string[] HighTokens = { "study_together", "library_note", "sports_cheer" };
/// &lt;summary&gt;One derived growth beat (the presentation ledger slice
/// B emits by count-diff; pure data, zero stat fx by design).&lt;/summary&gt;
public struct GrowthBeat
{
    public int stage;
    public int day;          // the school day the beat lands on
    public string npc;
    public string token;
}
/// &lt;summary&gt;The graduation-season confession resolution (dual
/// comedy endings, photo token for the album face).&lt;/summary&gt;
public struct ConfessionOutcome
{
    public string npc;
    public int value;
    public bool confessed;
    public string outcome;     // "confessed" / "friend_card"
    public string photoToken;  // the album photo token (slice B books it)
}
/// &lt;summary&gt;Guardrail 1 face: romance content unlocks at the middle
/// school stage - the two early stages stay pure friendship.&lt;/summary&gt;
public static bool RomanceBeatsAllowed(int stage)
    =&gt; stage &gt;= RomanceUnlockStage;
/// &lt;summary&gt;The growth beats booked by (stage, dayInYear) - the
/// frozen 30-day cadence, capped at three per stage; past stage 4
/// the earlier totals ride along, the graduate segment holds the
/// full six (the confession reads exactly this face).&lt;/summary&gt;
public static int GrowthBeatsSoFar(int stage, int dayInYear)
{
    if (stage &lt; RomanceUnlockStage) return 0;
    int total = 0;
    if (stage == 3 || stage == 4) total += Cap3(dayInYear);
    if (stage &gt;= 4) total += GrowthBeatsPerStage;
    if (stage &gt;= 5) total += GrowthBeatsPerStage;
    return total;
}
static int Cap3(int dayInYear)
{
    int n = dayInYear / GrowthCadenceDays;
    return n &gt; GrowthBeatsPerStage ? GrowthBeatsPerStage : n;
}
/// &lt;summary&gt;The derived growth ledger (stage-3 middle beats first,
/// then the stage-4 shy beats) - the slice-B emitter walks this by
/// count-diff, the checks assert it byte-exact.&lt;/summary&gt;
public static List&lt;GrowthBeat&gt; GrowthLedger(int stage, int dayInYear)
{
    var list = new List&lt;GrowthBeat&gt;();
    if (stage &lt; RomanceUnlockStage) return list;
    for (int s = RomanceUnlockStage; s &lt;= stage &amp;&amp; s &lt;= 4; s++)
```

```
        {
            int own = s == stage ? Cap3(dayInYear) : GrowthBeatsPerStage;
            string[] toks = s == 3 ? MiddleTokens : HighTokens;
            for (int i = 0; i &lt; own; i++)
                list.Add(new GrowthBeat
                {
                    stage = s,
                    day = (i + 1) * GrowthCadenceDays,
                    npc = TargetNpc,
                    token = toks[i % toks.Length],
                });
        }
        return list;
    }
    /// &lt;summary&gt;The T006 booking ledger formalized: per-NPC goodwill
    /// bumps over the log (the E04 chain tail books at the affinity_up
    /// step, the E09 open card books on the card). The seat mapping
    /// rides the CampusCodex frozen table - one mapping source, zero
    /// duplication.&lt;/summary&gt;
    public static int BookedFor(string npc, IList&lt;CampusEventLogEntry&gt; log)
    {
        if (npc == null || npc.Length == 0) return 0;
        int n = 0;
        if (log != null)
        {
            var defs = CampusData.Events();
            for (int i = 0; i &lt; log.Count; i++)
            {
                var e = log[i];
                if (CampusCodex.SeatOfEvent(e.id) != npc) continue;
                var def = DefOf(defs, e.id);
                if (def == null || def.affinity &lt;= 0) continue;
                if (e.kind == CampusEventKind.ChainStep)
                {
                    if (e.step == "affinity_up") n++;
                }
                else if (def.chain.Length == 0) n++;
            }
        }
        return n;
    }
    static EventDef DefOf(EventDef[] defs, string id)
    {
        for (int i = 0; i &lt; defs.Length; i++)
            if (defs[i].id == id) return defs[i];
        return null;
    }
    /// &lt;summary&gt;Per-NPC affinity value: the booked meetings plus the
    /// deskmate growth line (the other pool seats wait for their own
    /// future event beats).&lt;/summary&gt;
```

```
public static int ValueFor(string npc, IList&lt;CampusEventLogEntry&gt; log, int stage, int dayInYear)
    =&gt; BookedFor(npc, log) + (npc == TargetNpc ? GrowthBeatsSoFar(stage, dayInYear) : 0);
/// &lt;summary&gt;The confession target: the highest-value pool seat
/// (ties ride the frozen pool order).&lt;/summary&gt;
public static string ConfessionTarget(IList&lt;CampusEventLogEntry&gt; log, int stage, int dayInYear)
{
    string best = NpcPool[0];
    int bestV = int.MinValue;
    for (int i = 0; i &lt; NpcPool.Length; i++)
    {
        int v = ValueFor(NpcPool[i], log, stage, dayInYear);
        if (v &gt; bestV) { bestV = v; best = NpcPool[i]; }
    }
    return best;
}
/// &lt;summary&gt;The graduation-season confession (sect7: the
/// coming-of-age season): value at or over the threshold lands the
/// couple graduation photo, below it lands the friend-card comedy -
/// both endings book a photo and stay pure youth-memory flavor.&lt;/summary&gt;
public static ConfessionOutcome ResolveConfession(IList&lt;CampusEventLogEntry&gt; log, int stage, int dayInYear
{
    string npc = ConfessionTarget(log, stage, dayInYear);
    int v = ValueFor(npc, log, stage, dayInYear);
    bool ok = v &gt;= ConfessionThreshold;
    return new ConfessionOutcome
    {
        npc = npc,
        value = v,
        confessed = ok,
        outcome = ok ? "confessed" : "friend_card",
        photoToken = "GRAD PHOTO " + npc.ToUpperInvariant() + (ok ? " COUPLE" : " FRIEND CARD"),
    };
}
/// &lt;summary&gt;The slice-B confession beat detail (telemetry face,
/// machine-readable compliance trace).&lt;/summary&gt;
public static string ConfessionBeatDetail(ConfessionOutcome c)
    =&gt; "npc=" + c.npc + " value=" + c.value + " outcome=" + c.outcome;
/// &lt;summary&gt;Deterministic fingerprint over the whole affinity face
/// (the double-derive byte-parity instrument, zero-RNG family).&lt;/summary&gt;
public static string Sig(IList&lt;CampusEventLogEntry&gt; log, int stage, int dayInYear)
{
    var sb = new System.Text.StringBuilder();
    sb.Append("affinity:v1|booked=").Append(BookedFor(TargetNpc, log))
      .Append("|beats=").Append(GrowthBeatsSoFar(stage, dayInYear))
      .Append("|value=").Append(ValueFor(TargetNpc, log, stage, dayInYear));
    var ledger = GrowthLedger(stage, dayInYear);
    for (int i = 0; i &lt; ledger.Count; i++)
        sb.Append('|').Append(ledger[i].stage).Append('@').Append(ledger[i].day)
          .Append(':').Append(ledger[i].token);
    return sb.ToString();
```

```
        }
    }
}
// CampusAlbum.cs
// CampusAlbum.cs - the growth album data face (G08-T008 slice B).
// SYS05 downstream: every promotion and the graduation ending book one
// album entry here (the pixel growth-album is the visual signature - the
// ending review long image IS the self-propagation hammer). Pure data:
// the album survives StartNewJourney untouched (M08: the carry resets the
// spine, the memories stay), so a second journey appends after the first
// one's entries with its own journey stamp. The view renders Lines() and
// never re-derives; the checks assert exact bytes (the BoardLine family
// law - token telemetry face, localization rides the frame keys, not the
// tokens). Zero RNG, zero wall-clock, zero server faces.
using System.Collections.Generic;
using System.Text;
namespace Sy.CrazyCampus
{
    public sealed class CampusAlbum
    {
        /// &lt;summary&gt;One album entry: the journey stamp plus the token
        /// (BoardLine family: compact English telemetry, byte-stable).&lt;/summary&gt;
        public sealed class Entry
        {
            public int journey;
            public string kind = "";   // "stage" / "ending" / "photo"
            public string token = "";
        }
        public readonly List&lt;Entry&gt; Entries = new List&lt;Entry&gt;();
        /// &lt;summary&gt;The journey stamp - starts at 1, advances through
        /// StartJourney (the album itself never resets).&lt;/summary&gt;
        public int Journey { get; private set; }
        public CampusAlbum() { Journey = 1; }
        /// &lt;summary&gt;Save-restore factory (G08-T012): rebuilds the album at an
        /// exact captured state - the journey stamp plus every booked entry -
        /// so the memories survive a power cut byte-identically.&lt;/summary&gt;
        public static CampusAlbum Restore(int savedJourney, Entry[] savedEntries)
        {
            var a = new CampusAlbum();
            a.Journey = savedJourney &lt; 1 ? 1 : savedJourney;
            if (savedEntries != null)
                for (int i = 0; i &lt; savedEntries.Length; i++)
                    a.Entries.Add(savedEntries[i]);
            return a;
        }
        /// &lt;summary&gt;A promotion books one stage entry (the promoted stage
        /// reads the frozen stage table - the view/check never re-derive).&lt;/summary&gt;
        public void OnStageUp(int toStage)
        {
            Entries.Add(new Entry
```

```
    {
        journey = Journey,
        kind = "stage",
        token = "STAGE " + toStage + " " + CampusData.StageNames[toStage - 1],
    });
}
/// &lt;summary&gt;The graduation ending books the closing entry (the
/// fallback flag rides the token so the review can flag the comedy
/// of a lukewarm graduate).&lt;/summary&gt;
public void OnEnding(string endingId, bool matched)
{
    Entries.Add(new Entry
    {
        journey = Journey,
        kind = "ending",
        token = "ENDING " + (matched ? "" : "FALLBACK ") + endingId.ToUpperInvariant(),
    });
}
/// &lt;summary&gt;G08-T013 slice B: the graduation-season confession books
/// one photo after the ending (the SYS06 dual comedy endings - both
/// outcomes land a photo, the token carries the outcome). The photo
/// kind rides Sig() (dynamic byte-parity) but Lines() skips it: the
/// long-image and share-card byte faces stay frozen.&lt;/summary&gt;
public void AddPhoto(string photoToken)
{
    Entries.Add(new Entry
    {
        journey = Journey,
        kind = "photo",
        token = photoToken,
    });
}
/// &lt;summary&gt;M08 second journey: only the stamp advances - the
/// earlier memories are the kept-album law, never cleared.&lt;/summary&gt;
public void StartJourney() { Journey++; }
/// &lt;summary&gt;The long-image token face (the review lines the ending
/// overlay and the album panel render verbatim). The photo kind
/// (T013 slice B) never enters the long image - the byte faces the
/// checks froze stay frozen; the photos live in Sig() only.&lt;/summary&gt;
public string Lines()
{
    var sb = new StringBuilder();
    for (int i = 0; i &lt; Entries.Count; i++)
    {
        if (Entries[i].kind == "photo") continue;
        if (sb.Length &gt; 0) sb.Append(" | ");
        sb.Append(Entries[i].token);
    }
    return sb.ToString();
}
```

```
        /// &lt;summary&gt;Byte-parity instrument: identical drives land on
        /// identical album bytes (the zero-RNG family proof).&lt;/summary&gt;
        public string Sig()
        {
            var sb = new StringBuilder();
            sb.Append("J").Append(Journey).Append('#');
            for (int i = 0; i &lt; Entries.Count; i++)
                sb.Append(Entries[i].journey).Append(':')
                  .Append(Entries[i].kind).Append(':').Append(Entries[i].token).Append(';');
            return sb.ToString();
        }
    }
}
// CampusAudioCatalog.cs
// CampusAudioCatalog.cs - generated-audio catalog (G08 audio wiring prep,
// batch B642). X027-isomorphic to the G13 FarmAudioCatalog paradigm: the
// CampusAudioSetup editor tool scans Assets/_Game/Art/Audio and serializes
// clip references here so runtime code can resolve audio by filename stem.
// The asset lives at Assets/_Game/Resources/CampusAudioCatalog.asset
// (Resources keeps the playtest-slice lookup build-safe with zero
// Addressables wiring, same rationale as ArtCatalog). Missing clip = null =
// stay silent (X027 null-safe law): headless gates stay green whatever the
// import state is - the cloudF wave (B637, 4 stems pending) lands later and
// the catalog rebuild lights the whole face up with zero code churn.
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.CrazyCampus
{
    [CreateAssetMenu(fileName = "CampusAudioCatalog", menuName = "Sy/CrazyCampus Audio Catalog")]
    public sealed class CampusAudioCatalog : ScriptableObject
    {
        [Serializable]
        public sealed class Entry
        {
            public string key;      // filename stem, e.g. "g08_hook_bgm"
            public AudioClip clip; // direct reference keeps builds self-contained
        }
        public List&lt;Entry&gt; entries = new List&lt;Entry&gt;();
        Dictionary&lt;string, AudioClip&gt; _index;
        void OnEnable()
        {
            // Serializer revival can pair a fresh instance with a stale cache;
            // drop it so the next Get() rebuilds from entries.
            _index = null;
        }
        public AudioClip Get(string key)
        {
            if (string.IsNullOrEmpty(key)) return null;
            if (_index == null)
```

```
        {
            _index = new Dictionary&lt;string, AudioClip&gt;(entries.Count, StringComparer.Ordinal);
            for (int i = 0; i &lt; entries.Count; i++)
            {
                var e = entries[i];
                if (e != null &amp;&amp; !string.IsNullOrEmpty(e.key) &amp;&amp; e.clip != null &amp;&amp; !_index.C
                    _index[e.key] = e.clip;
            }
        }
        return _index.TryGetValue(key, out var c) ? c : null;
    }
}
/// &lt;summary&gt;
/// Runtime lookup helper (FarmAudio mirror). Consumers MUST treat null as
/// "audio missing -&gt; stay silent" so headless checks stay green on any
/// project state (catalog absent, clip pruned, name typo). Stem constants
/// are the single consumption contract both sides code against - they
/// match the cloudF-B-AUDIO-B637 live-queue ids one-to-one.
/// &lt;/summary&gt;
public static class CampusAudio
{
    // cloudF-B-AUDIO-B637 stems (audio-book G08 row): the loopable hook
    // BGM bed plus the three same-tick feedback one-shots (R3 face).
    public const string HookBgm = "g08_hook_bgm";
    public const string SfxFirstGrowth = "g08_sfx_first_growth";
    public const string SfxEventCard = "g08_sfx_event_card";
    public const string SfxStageUp = "g08_sfx_stage_up";
    static CampusAudioCatalog _catalog;
    public static CampusAudioCatalog Catalog
    {
        get
        {
            if (_catalog == null) _catalog = Resources.Load&lt;CampusAudioCatalog&gt;("CampusAudioCatalog");
            return _catalog;
        }
    }
    public static AudioClip Get(string key)
    {
        var c = Catalog;
        return c != null ? c.Get(key) : null;
    }
    /// &lt;summary&gt;
    /// Loop the requested stem on a fresh host object and return the source.
    /// Missing clip resolves to silence (returns null) - never an error, so
    /// the playtest slice runs with or without the audio wave landed.
    /// &lt;/summary&gt;
    public static AudioSource PlayLoop(string key, float volume = 0.6f)
    {
        var clip = Get(key);
        if (clip == null) return null;
```

```
            var go = new GameObject("CampusAudio(" + key + ")");
            var src = go.AddComponent&lt;AudioSource&gt;();
            src.clip = clip;
            src.loop = true;
            src.volume = volume;
            src.Play();
            return src;
        }
        /// &lt;summary&gt;
        /// Fire a one-shot feedback stem. Missing clip = silence (no object
        /// spawned, no error) so event-driven wiring is safe pre-arrival.
        /// &lt;/summary&gt;
        public static void PlayOneShot(string key, float volume = 0.8f)
        {
            var clip = Get(key);
            if (clip == null) return;
            var go = new GameObject("CampusSfx(" + key + ")");
            var src = go.AddComponent&lt;AudioSource&gt;();
            src.clip = clip;
            src.loop = false;
            src.volume = volume;
            src.Play();
            // static class has no UnityEngine.Object inheritance face, the
            // destroy call must be fully qualified (X924 non-MonoBehaviour law).
            UnityEngine.Object.Destroy(go, clip.length + 0.05f);
        }
    }
}
// CampusAudioHost.cs
// CampusAudioHost.cs - S08_Main runtime hook for the generated-audio face
// (G08 audio wiring prep, batch B642). Mirror of FarmAudioHost: the
// scene-side face that starts the looping hook BGM bed on play start.
// CampusAudio.PlayLoop is null-safe by contract (missing catalog/clip =
// silence, never an error), so the headless gates stay green whatever the
// import state is - wired by ProjectSetup.EnsureAudioHost (G13 B583 law).
using UnityEngine;
namespace Sy.CrazyCampus
{
    public sealed class CampusAudioHost : MonoBehaviour
    {
        void Start()
        {
            CampusAudio.PlayLoop(CampusAudio.HookBgm);
        }
    }
}
// CampusCodex.cs
// CampusCodex.cs - the classmate &amp;amp; event codex data faces (G08-T010 slice B).
// Design doc section 8 is the single consumption source: the classmate codex
// derives from the event engine's CARD beats through the frozen cast mapping
```

```
// (the engine log carries no actor field - the section 8.2 mapping IS the
// actor history), the event codex aggregates the log directly. Unmapped ids
// (future event tables) land in the mystery seat - never a crash, never a
// silent drop (the consumption contract). Derive-on-read: zero new game
// state, zero RNG, zero wall-clock, zero server faces; the slice-C tab UI
// consumes Seats()/Lines() and never re-derives.
using System.Collections.Generic;
using System.Text;
namespace Sy.CrazyCampus
{
    public sealed class CampusCodex
    {
        /// &lt;summary&gt;One cast seat (design doc 8.1, the P1 art face stems):
        /// meme token + niche line + the mapped event ids. English telemetry
        /// face (BoardLine family: byte-stable; zh display keys land slice C).&lt;/summary&gt;
        public sealed class SeatDef
        {
            public string key = "";
            public string name = "";
            public string niche = "";
            public string[] events = new string[0];
        }
        /// &lt;summary&gt;The unmapped-id bucket seat (the contract fallback).&lt;/summary&gt;
        public const string MysterySeatKey = "mystery";
        public const string MysterySeatName = "MYSTERY CLASSMATE";
        // ---- frozen cast table (10 seats, design doc 8.1 order) ----
        // swot/inventor carry zero E-mappings by design: their beats are the
        // system faces (exam M03 pair / S2 club slot) that wire in later -
        // the seats ride the table undiscovered until then, never a crash.
        static readonly SeatDef[] CastTable =
        {
            new SeatDef { key = "deskmate", name = "DESKMATE RULER LINE",   niche = "the desk-border ruler patrol",    e
            new SeatDef { key = "swot",     name = "SWOT FULL-MARK HALO",   niche = "the glowing full-mark paper",     e
            new SeatDef { key = "sleeper",  name = "SLEEPER DESK DROOL",    niche = "the classroom sleep-pod regular", e
            new SeatDef { key = "foodie",   name = "FOODIE LUNCH-BOX TOWER", niche = "the perpetual lunch-box tower",
            new SeatDef { key = "athlete",  name = "ATHLETE PINNY SPRINT",  niche = "the pinny that never comes off",  e
            new SeatDef { key = "artist",   name = "ARTIST TEXTBOOK DOODLE", niche = "the textbook-corner serial artist"
            new SeatDef { key = "singer",   name = "SINGER OFF-KEY ANTHEM", niche = "the self-lost off-key anthem",    e
            new SeatDef { key = "gossip",   name = "GOSSIP COMIC CIRCULATION", niche = "the underground comic circulatio
            new SeatDef { key = "lucky",    name = "LUCKY C-ALL-RIGHT",     niche = "the all-C miracle luck",          e
            new SeatDef { key = "inventor", name = "INVENTOR SMOKING GADGET", niche = "the smoking-gadget lab risk",   e
        };
        public static SeatDef[] Seats() =&gt; (SeatDef[])CastTable.Clone();
        /// &lt;summary&gt;The seat a frozen event maps to ("" = unmapped = the
        /// mystery bucket at derive time). Never throws.&lt;/summary&gt;
        public static string SeatOfEvent(string eventId)
        {
            for (int i = 0; i &lt; CastTable.Length; i++)
                for (int j = 0; j &lt; CastTable[i].events.Length; j++)
                    if (CastTable[i].events[j] == eventId) return CastTable[i].key;
```

```
    return "";
}
/// &lt;summary&gt;Frozen cast signature - the schema-freeze proof literal
/// (the check asserts the exact bytes; the table only moves with a
/// design-doc section 8 revision).&lt;/summary&gt;
public static string CastSig()
{
    var sb = new StringBuilder();
    sb.Append("cast:v1");
    for (int i = 0; i &lt; CastTable.Length; i++)
    {
        sb.Append('|').Append(CastTable[i].key).Append('=');
        for (int j = 0; j &lt; CastTable[i].events.Length; j++)
        {
            if (j &gt; 0) sb.Append('+');
            sb.Append(CastTable[i].events[j]);
        }
    }
    return sb.ToString();
}
/// &lt;summary&gt;Event codex token (design doc 8.2 four columns: type /
/// fx / chain / affinity, plus the stage scope) - a pure read of the
/// frozen table, "" for unknown ids.&lt;/summary&gt;
public static string EventTokenOf(string eventId)
{
    var table = CampusData.Events();
    for (int i = 0; i &lt; table.Length; i++)
    {
        if (table[i].id != eventId) continue;
        var d = table[i];
        var sb = new StringBuilder();
        sb.Append(d.id).Append('|').Append(d.type).Append("|fx=")
          .Append(d.fx.intell).Append(',').Append(d.fx.vit).Append(',')
          .Append(d.fx.charm).Append(',').Append(d.fx.mood).Append(',')
          .Append(d.fx.pocket);
        if (d.chain.Length &gt; 0) sb.Append("|chain=").Append(d.chain);
        if (d.affinity != 0) sb.Append("|affinity=").Append(d.affinity);
        sb.Append("|stage=").Append(d.stage);
        return sb.ToString();
    }
    return "";
}
// ---- derived instance (built by Derive, never mutated after) ----
readonly Dictionary&lt;string, int&gt; seatCount = new Dictionary&lt;string, int&gt;();
readonly Dictionary&lt;string, double&gt; seatFirst = new Dictionary&lt;string, double&gt;();
readonly Dictionary&lt;string, int&gt; eventCount = new Dictionary&lt;string, int&gt;();
readonly Dictionary&lt;string, double&gt; eventFirst = new Dictionary&lt;string, double&gt;();
readonly Dictionary&lt;string, int&gt; mysteryPerId = new Dictionary&lt;string, int&gt;();
readonly List&lt;string&gt; mysteryOrder = new List&lt;string&gt;();
int totalCards;
```

```
/// &lt;summary&gt;The derivation law (the consumption contract): scan the
/// engine log's card beats, map each id through the frozen cast,
/// count the appearances and keep the first-seen virtual second.
/// Chain steps are the event's own beats - the appearance history
/// counts cards only. Null-safe and empty-safe.&lt;/summary&gt;
public static CampusCodex Derive(IList&lt;CampusEventLogEntry&gt; log)
{
    var c = new CampusCodex();
    if (log == null) return c;
    for (int i = 0; i &lt; log.Count; i++)
    {
        var e = log[i];
        if (e == null || e.kind != CampusEventKind.Card) continue;
        c.totalCards++;
        if (!c.eventCount.ContainsKey(e.id))
        {
            c.eventCount[e.id] = 0;
            c.eventFirst[e.id] = e.t;
        }
        c.eventCount[e.id]++;
        string seat = SeatOfEvent(e.id);
        if (seat.Length == 0)
        {
            seat = MysterySeatKey;
            if (!c.mysteryPerId.ContainsKey(e.id))
            {
                c.mysteryPerId[e.id] = 0;
                c.mysteryOrder.Add(e.id);
            }
            c.mysteryPerId[e.id]++;
        }
        if (!c.seatCount.ContainsKey(seat))
        {
            c.seatCount[seat] = 0;
            c.seatFirst[seat] = e.t;
        }
        c.seatCount[seat]++;
    }
    return c;
}
public static CampusCodex Derive(CampusEventEngine engine)
    =&gt; Derive(engine == null ? null : engine.Log);
public int TotalCards =&gt; totalCards;
public int MysteryCount =&gt; mysteryOrder.Count;
public IList&lt;string&gt; MysteryIds =&gt; mysteryOrder;
public int AppearancesOf(string seatKey)
    =&gt; seatCount.TryGetValue(seatKey, out int n) ? n : 0;
public double FirstSeenOf(string seatKey)
    =&gt; seatFirst.TryGetValue(seatKey, out double t) ? t : -1.0;
public bool Discovered(string seatKey) =&gt; AppearancesOf(seatKey) &gt; 0;
```

```
public int EventCountOf(string eventId)
    =&gt; eventCount.TryGetValue(eventId, out int n) ? n : 0;
public double EventFirstOf(string eventId)
    =&gt; eventFirst.TryGetValue(eventId, out double t) ? t : -1.0;
/// &lt;summary&gt;Sum of the mapped-seat appearance counts (the mystery
/// bucket rides its own face).&lt;/summary&gt;
public int SeatFacesSum()
{
    int sum = 0;
    for (int i = 0; i &lt; CastTable.Length; i++)
        sum += AppearancesOf(CastTable[i].key);
    return sum;
}
/// &lt;summary&gt;The no-drop contract law: every card beat lands in
/// exactly one face - a mapped seat or the mystery bucket.&lt;/summary&gt;
public bool NoDropLaw =&gt; SeatFacesSum() + AppearancesOf(MysterySeatKey) == totalCards;
/// &lt;summary&gt;Panel token face: one line per discovered seat in cast
/// order (the slice-C tabs render verbatim; locked seats derive from
/// the table, not from this face).&lt;/summary&gt;
public string Lines()
{
    var sb = new StringBuilder();
    for (int i = 0; i &lt; CastTable.Length; i++)
    {
        int n = AppearancesOf(CastTable[i].key);
        if (n &lt;= 0) continue;
        if (sb.Length &gt; 0) sb.Append(" | ");
        sb.Append(CastTable[i].name).Append(" x").Append(n)
          .Append(" first@").Append(FirstSeenOf(CastTable[i].key).ToString("0.###"));
    }
    int mystery = AppearancesOf(MysterySeatKey);
    if (mystery &gt; 0)
    {
        if (sb.Length &gt; 0) sb.Append(" | ");
        sb.Append(MysterySeatName).Append(" x").Append(mystery);
    }
    return sb.ToString();
}
// ---- slice-C tab faces (BoardLine family: byte-stable tokens the
// tab bodies render verbatim). A locked seat keeps the ??? silhouette
// plus its niche tease (the collection to-do); a discovered seat
// carries the name, count, first-seen beat and niche. ----
public const string LockedSeatToken = "???";
public const string EventUnseenToken = "NOT SEEN YET";
/// &lt;summary&gt;One classmate line: the discovered face appends the
/// niche; the locked face keeps the silhouette and the tease.&lt;/summary&gt;
public string SeatLine(SeatDef seat)
{
    int n = AppearancesOf(seat.key);
    return n &gt; 0
```

```
        ? seat.name + " x" + n + " first@" + FirstSeenOf(seat.key).ToString("0.###")
          + " - " + seat.niche
        : LockedSeatToken + " - " + seat.niche;
}
/// &lt;summary&gt;Classmates tab body: every seat rides the cast order
/// (locked included - the codex is a collection list, not a log);
/// the mystery bucket closes the face when present.&lt;/summary&gt;
public string SeatsFace()
{
    var sb = new StringBuilder();
    for (int i = 0; i &lt; CastTable.Length; i++)
    {
        if (sb.Length &gt; 0) sb.Append(" | ");
        sb.Append(SeatLine(CastTable[i]));
    }
    int mystery = AppearancesOf(MysterySeatKey);
    if (mystery &gt; 0)
        sb.Append(" | ").Append(MysterySeatName).Append(" x").Append(mystery);
    return sb.ToString();
}
/// &lt;summary&gt;Events tab body: one line per frozen event in table
/// order - the seen face carries the count, first-seen beat and
/// type column; the unseen face keeps the id and the tease.&lt;/summary&gt;
public string EventsFace()
{
    var sb = new StringBuilder();
    var table = CampusData.Events();
    for (int i = 0; i &lt; table.Length; i++)
    {
        if (sb.Length &gt; 0) sb.Append(" | ");
        int n = EventCountOf(table[i].id);
        if (n &gt; 0)
            sb.Append(table[i].id).Append(" x").Append(n)
              .Append(" first@").Append(EventFirstOf(table[i].id).ToString("0.###"))
              .Append(" - ").Append(table[i].type);
        else
            sb.Append(table[i].id).Append(" - ").Append(EventUnseenToken)
              .Append(" - ").Append(table[i].type);
    }
    return sb.ToString();
}
/// &lt;summary&gt;Derivation fingerprint - the double-run byte-parity face
/// (zero-RNG family): identical logs derive identical bytes.&lt;/summary&gt;
public string Sig()
{
    var sb = new StringBuilder();
    sb.Append("cards=").Append(totalCards);
    for (int i = 0; i &lt; CastTable.Length; i++)
        sb.Append('|').Append(CastTable[i].key).Append('=')
          .Append(AppearancesOf(CastTable[i].key)).Append('@')
```

```
                .Append(FirstSeenOf(CastTable[i].key).ToString("0.###"));
            var table = CampusData.Events();
            for (int i = 0; i &lt; table.Length; i++)
                sb.Append('|').Append(table[i].id).Append('=')
                  .Append(EventCountOf(table[i].id)).Append('@')
                  .Append(EventFirstOf(table[i].id).ToString("0.###"));
            sb.Append("|mystery=").Append(AppearancesOf(MysterySeatKey));
            for (int i = 0; i &lt; mysteryOrder.Count; i++)
                sb.Append('|').Append(mysteryOrder[i]).Append('=').Append(mysteryPerId[mysteryOrder[i]]);
            return sb.ToString();
        }
    }
}
// CampusCostumePool.cs
// CampusCostumePool.cs - the costume &amp; title collection pool (G08-T015
// slice D, the numeric pack section 12 wiring order tail: the ten-pull pool
// data layer the g08_costume_pull Direct 1/day seat draws from).
// Pure collection face: one pull grants student outfits and cast-seat
// titles - ZERO growth numbers ride the pool (U091 zero-dispute seat: no
// StatDelta anywhere in this file, proven by the gate, not trusted from
// the shape). Deterministic zero-RNG family: the pull sequence is a pure
// function of the pull cursor (double-run byte-parity law) - identical
// cursors grant identical batches forever, no wall-clock, no server faces
// (red line 12). Duplicates deepen the collection depth only - no shard
// economy, no stat compensation (the honest dup law). The pool composes
// frozen: 30 outfits (5 school stages x 6 look variants) + 30 titles
// (10 cast seats x 3 tiers) = 60 seats of collection desire; the cast
// cross-face rides the codex seats (the section 12 friction face three:
// collection wants the ensemble, the ensemble lives in CampusCodex).
using System;
using System.Text;
namespace Sy.CrazyCampus
{
    /// &lt;summary&gt;One pool seat: an outfit or a cast title. Pure data, pure
    /// English telemetry face (BoardLine family).&lt;/summary&gt;
    public sealed class CostumeDef
    {
        public string id = "";
        public string kind = "";     // outfit / title
        public string name = "";
        public string tease = "";     // the locked-face tease line
    }
    /// &lt;summary&gt;The frozen pool table + the deterministic pull law (the
    /// rules ARE the table).&lt;/summary&gt;
    public static class CampusCostumeTable
    {
        public const int PoolCount = 60;          // 30 outfits + 30 titles
        public const int OutfitCount = 30;        // 5 stages x 6 variants
        public const int TitleCount = 30;         // 10 seats x 3 tiers
        public const int TenPullSize = 10;        // one rewarded view = ten grants
```

```
public const int StageSpan = 5;           // StageCount mirror (frozen pair)
public const int VariantCount = 6;
public const int SeatSpan = 10;          // CampusCodex cast mirror
public const int TierCount = 3;
// ---- frozen composition parts ----
static readonly string[] OutfitVariants =
    { "SPORTS", "FESTIVAL", "EXAM-WEEK", "RAINCOAT", "PE JERSEY", "CARDIGAN" };
static readonly string[] OutfitTeases =
{
    "the track-day badge look", "the parade-day confetti look",
    "the armor-for-the-battle look", "the puddle-season armor look",
    "the house-team colors look", "the quiet-corner reader look",
};
static readonly string[] TierTokens = { "ROOKIE", "SENIOR", "LEGEND" };
static readonly string[] TierTeases =
{
    "the first-year earned stripe", "the seasoned veteran stripe",
    "the whole-corridor-knows stripe",
};
static readonly CostumeDef[] PoolTable = BuildTable();
static CostumeDef[] BuildTable()
{
    var t = new CostumeDef[PoolCount];
    int n = 0;
    var stages = CampusData.StageNames;
    for (int s = 0; s &lt; StageSpan; s++)
    {
        for (int v = 0; v &lt; VariantCount; v++)
        {
            t[n] = new CostumeDef
            {
                id = "out_s" + (s + 1) + "_v" + (v + 1),
                kind = "outfit",
                name = stages[s] + " " + OutfitVariants[v] + " SUIT",
                tease = OutfitTeases[v],
            };
            n++;
        }
    }
    var seats = CampusCodex.Seats();
    for (int k = 0; k &lt; SeatSpan; k++)
    {
        for (int r = 0; r &lt; TierCount; r++)
        {
            t[n] = new CostumeDef
            {
                id = "tit_" + seats[k].key + "_t" + (r + 1),
                kind = "title",
                name = seats[k].name + " " + TierTokens[r],
                tease = TierTeases[r] + " of the " + seats[k].niche,
```

```
            };
            n++;
        }
    }
    return t;
}
public static CostumeDef[] Pool() =&gt; (CostumeDef[])PoolTable.Clone();
public static CostumeDef DefOf(int index)
    =&gt; index &gt;= 0 &amp;&amp; index &lt; PoolCount ? PoolTable[index] : null;
/// &lt;summary&gt;Index of an id (-1 = unknown, never a crash).&lt;/summary&gt;
public static int IndexOf(string id)
{
    for (int i = 0; i &lt; PoolCount; i++)
        if (PoolTable[i].id == id) return i;
    return -1;
}
/// &lt;summary&gt;The deterministic pull slot for batch position v (the
/// zero-RNG law: a double-mixed multiplicative hash of the position
/// - pure int math, culture-safe, byte-stable on every machine).&lt;/summary&gt;
public static int SlotOf(int v)
{
    uint h = (uint)v * 2654435761u;
    h ^= h &gt;&gt; 15;
    h *= 2246822519u;
    h ^= h &gt;&gt; 13;
    return (int)(h % (uint)PoolCount);
}
/// &lt;summary&gt;The ten grants of one pull from a cursor: positions
/// cursor*TenPullSize .. +9 mapped through SlotOf. Identical cursors
/// produce identical batches, forever.&lt;/summary&gt;
public static int[] TenPullBatch(int cursor)
{
    var b = new int[TenPullSize];
    for (int i = 0; i &lt; TenPullSize; i++)
        b[i] = SlotOf(cursor * TenPullSize + i);
    return b;
}
/// &lt;summary&gt;Frozen table signature (the schema-freeze face; byte-stable
/// by construction, asserted double-derive by the gate).&lt;/summary&gt;
public static string PoolSig()
{
    var sb = new StringBuilder();
    sb.Append("pool:v1|n=").Append(PoolCount);
    int outfits = 0, titles = 0;
    for (int i = 0; i &lt; PoolCount; i++)
    {
        sb.Append('|').Append(PoolTable[i].id).Append('=')
          .Append(PoolTable[i].kind).Append(',').Append(PoolTable[i].name);
        if (PoolTable[i].kind == "outfit") outfits++; else titles++;
    }
```

```
            sb.Append("|split=").Append(outfits).Append(',').Append(titles);
            return sb.ToString();
        }
    }
    /// &lt;summary&gt;The owned-collection state machine over the frozen pool.
    /// Pull cursor + per-seat first-grant ordinal + dup depth. Capture/
    /// Restore round-trips the whole state so a later save wire-in is a
    /// three-line lift (slice E face).&lt;/summary&gt;
    public sealed class CampusCostumePool
    {
        int pulls;                      // the pull cursor (batches taken)
        readonly int[] firstAt;         // -1 = not owned, else the grant ordinal of first ownership
        readonly int[] dups;            // duplicate depth per seat
        public CampusCostumePool()
        {
            firstAt = new int[CampusCostumeTable.PoolCount];
            dups = new int[CampusCostumeTable.PoolCount];
            for (int i = 0; i &lt; firstAt.Length; i++) firstAt[i] = -1;
        }
        public int Pulls =&gt; pulls;
        public int PoolSize =&gt; CampusCostumeTable.PoolCount;
        public int OwnedCount { get; private set; }
        public double CollectionPercent =&gt; OwnedCount * 100.0 / CampusCostumeTable.PoolCount;
        public bool Owns(int index) =&gt; index &gt;= 0 &amp;&amp; index &lt; firstAt.Length &amp;&amp; firstAt[index] &
        public int DupsOf(int index) =&gt; index &gt;= 0 &amp;&amp; index &lt; dups.Length ? dups[index] : 0;
        public int FirstAt(int index) =&gt; index &gt;= 0 &amp;&amp; index &lt; firstAt.Length ? firstAt[index] : -1;
        /// &lt;summary&gt;One ten-pull: grants the deterministic batch of the next
        /// cursor position. New seats become owned (first-grant ordinal
        /// booked), already-owned seats deepen their dup count. Returns the
        /// granted ids in draw order (the face a later ad-complete lands
        /// through; the gate drives it directly - hermetic, no ad service in
        /// the data layer).&lt;/summary&gt;
        public string[] GrantTen()
        {
            int[] batch = CampusCostumeTable.TenPullBatch(pulls);
            var ids = new string[CampusCostumeTable.TenPullSize];
            int ordinal = pulls * CampusCostumeTable.TenPullSize;
            for (int i = 0; i &lt; batch.Length; i++)
            {
                int seat = batch[i];
                ids[i] = CampusCostumeTable.DefOf(seat).id;
                if (firstAt[seat] &lt; 0) { firstAt[seat] = ordinal + i; OwnedCount++; }
                else dups[seat]++;
            }
            pulls++;
            return ids;
        }
        /// &lt;summary&gt;Whole-state capture (the save seam face).&lt;/summary&gt;
        public PoolState Capture()
        {
```

```
            var st = new PoolState
            {
                pulls = this.pulls,
                ownedCount = this.OwnedCount,
                firstAt = (int[])this.firstAt.Clone(),
                dups = (int[])this.dups.Clone(),
            };
            return st;
        }
        public static CampusCostumePool Restore(PoolState? st)
        {
            var p = new CampusCostumePool();
            if (st == null || st.Value.firstAt == null) return p;
            int n = CampusCostumeTable.PoolCount;
            p.pulls = st.Value.pulls &lt; 0 ? 0 : st.Value.pulls;
            for (int i = 0; i &lt; n &amp;&amp; i &lt; st.Value.firstAt.Length; i++) p.firstAt[i] = st.Value.firstAt[i];
            for (int i = 0; i &lt; n &amp;&amp; i &lt; st.Value.dups.Length; i++) p.dups[i] = st.Value.dups[i];
            int owned = 0;
            for (int i = 0; i &lt; n; i++) if (p.firstAt[i] &gt;= 0) owned++;
            p.OwnedCount = owned;
            return p;
        }
        public struct PoolState
        {
            public int pulls;
            public int ownedCount;      // derived on restore, kept for cross-checks
            public int[] firstAt;
            public int[] dups;
        }
        /// &lt;summary&gt;Derivation fingerprint - the double-run byte-parity face
        /// (identical pull histories produce identical bytes).&lt;/summary&gt;
        public string Sig()
        {
            var sb = new StringBuilder();
            sb.Append("pulls=").Append(pulls).Append("|owned=").Append(OwnedCount);
            for (int i = 0; i &lt; firstAt.Length; i++)
                sb.Append('|').Append(i).Append(':').Append(firstAt[i]).Append('/').Append(dups[i]);
            return sb.ToString();
        }
        /// &lt;summary&gt;Collection face token (the later tab UI renders this
        /// verbatim; BoardLine family).&lt;/summary&gt;
        public string CollectionFace()
        {
            return "COLLECTION " + OwnedCount + "/" + PoolSize
                + " (" + CollectionPercent.ToString("0.#") + "%) pulls=" + pulls;
        }
    }
}
// CampusCostumePull.cs
// CampusCostumePull.cs - the costume ten-pull ad face (G08-T015 slice E,
```

```
// the numeric pack section 12 wiring order tail: the g08_costume_pull
// Direct 1/day seat over the slice-D pool). Collection-only face: one
// completed rewarded view grants the pool's deterministic ten-pull batch -
// zero growth numbers ride the point (U091 zero-dispute seat, the pool file
// owns that proof), the stage spine stays untouched by the claim flow. The
// T009 three-piece law rides verbatim: HasCategoryBudget precheck and the
// ShowRewarded consumption share one policy instance and one day bucket (they
// can never disagree), the evt_ad funnel books offer -&gt; show -&gt; complete -&gt;
// costume_ten in order, and a capped tap refuses BEFORE the ad ever plays. A
// failed view burns nothing on the pool: no grant, cursor frozen, no
// costume_ten entry (the view itself is honestly consumed - a played ad is
// a played ad). No stage dependency - the collection face has no dead tap;
// no wall-clock, zero RNG, red line 12 whole (funnel entries stay bare so
// the double-run byte-parity family holds on the funnel string itself).
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Platform;
namespace Sy.CrazyCampus
{
    /// &lt;summary&gt;IAA second point (T015 slice E): the ten-pull collection
    /// face over CampusCostumePool. Deterministic for identical inputs.&lt;/summary&gt;
    public sealed class CampusCostumePull
    {
        /// &lt;summary&gt;Costume pull point id (T015 slice B mapping): draws from
        /// the shared direct 1/day category (AdCategories.CategoryOfPoint
        /// authority, the g08_ prefix keeps id namespaces isolated).&lt;/summary&gt;
        public const string CostumePullPoint = "g08_costume_pull";
        readonly CampusCostumePool pool;
        readonly AdCapPolicy adCap;
        readonly IAdsService ads;
        readonly Func&lt;string&gt; dayKey;
        public CampusCostumePull(CampusCostumePool pool, AdCapPolicy adCap, IAdsService ads, Func&lt;string&gt; dayKey =
        {
            this.pool = pool;
            this.adCap = adCap;
            this.ads = ads;
            // the precheck reads the very bucket the registered service consumes
            // (injected key only for deterministic gates; production rides TodayKey)
            this.dayKey = dayKey ?? AdCapPolicy.TodayKey;
            if (ads != null) ads.OnAdEvent += OnAdLifecycle;
        }
        /// &lt;summary&gt;Live direct-category precheck: always-fresh budget face
        /// on the same policy instance and day bucket the registered service
        /// consumes from, so precheck and consume can never disagree.&lt;/summary&gt;
        public bool OfferBudgetOk
        {
            get
            {
                return adCap != null
```

```
                    &amp;&amp; adCap.HasCategoryBudget(dayKey(), AdCategories.CategoryOfPoint(CostumePullPoint));
            }
        }
        /// &lt;summary&gt;Ten-pull CTA: budget precheck - rewarded view - the
        /// deterministic batch grants. The grant only ever lands when the
        /// view completed (a failed view leaves the cursor frozen for a
        /// retry). A capped tap books the honest refusal BEFORE the ad ever
        /// plays.&lt;/summary&gt;
        public async Task&lt;bool&gt; ClaimTenPull()
        {
            if (pool == null) return false;
            if (!OfferBudgetOk)
            {
                Funnel.Add("evt_ad_offer_capped");
                return false;
            }
            Funnel.Add("evt_ad_offer");
            RewardResult r = await ads.ShowRewarded(CostumePullPoint);
            if (r == null || !r.Completed) return false;   // evt_ad_fail rides the lifecycle face; the cursor stays fro
            pool.GrantTen();
            Funnel.Add("evt_costume_ten");
            return true;
        }
        /// &lt;summary&gt;evt_ad funnel (T009 law): lifecycle events of the
        /// costume point land in the funnel list for gate assertions.&lt;/summary&gt;
        void OnAdLifecycle(AdLifecycle e)
        {
            if (e.PointId != CostumePullPoint) return;
            Funnel.Add("evt_ad_" + e.Type.ToString().ToLowerInvariant());
        }
        public readonly List&lt;string&gt; Funnel = new List&lt;string&gt;();
    }
}
// CampusData.cs
// CampusData.cs - frozen data schemas and tables for Crazy Campus (G08-T004).
// The S0 stat pack (the G08 numeric pack doc v1, the doc package) is the
// source of truth and this file is its compiled mirror: schema freeze means
// the structural laws live here as code (the family WaveTable law - the rules
// ARE the table) and DataSchemaCheck proves every law on every row. Values
// are sweep-tunable later (T005 bot sweeps re-freeze revisions through the
// doc pack), the structure only moves with a schema version bump. Zero
// server faces: the tables ship in the build (red line 12).
namespace Sy.CrazyCampus
{
    /// &lt;summary&gt;The weight-disc quartet: study/play/club/rest shares of one
    /// game-day. Schema laws: each slot 0-100, the four slots sum to exactly
    /// 100 (Normalize enforces it by largest remainder, zero-input falls to
    /// the balanced default).&lt;/summary&gt;
    public struct ScheduleWeights
    {
```

```
    public int study;
    public int play;
    public int club;
    public int rest;
    public int Sum() =&gt; study + play + club + rest;
    /// &lt;summary&gt;The balanced default: a quarter day each.&lt;/summary&gt;
    public static ScheduleWeights Default()
    {
        var w = new ScheduleWeights { study = 25, play = 25, club = 25, rest = 25 };
        return w;
    }
    /// &lt;summary&gt;Clamps every slot into 0-100 and rescales to a sum of
    /// exactly 100 by largest remainder (a zero or empty disc falls back
    /// to the balanced default - the idle student never starves).&lt;/summary&gt;
    public static ScheduleWeights Normalized(ScheduleWeights w)
    {
        int a = Clamp(w.study), b = Clamp(w.play), c = Clamp(w.club), d = Clamp(w.rest);
        int sum = a + b + c + d;
        if (sum &lt;= 0) return Default();
        int[] v = { a, b, c, d };
        int[] scaled = new int[4];
        for (int i = 0; i &lt; 4; i++) scaled[i] = v[i] * 100 / sum;
        int left = 100 - (scaled[0] + scaled[1] + scaled[2] + scaled[3]);
        while (left &gt; 0)
        {
            // hand each remainder unit to the current largest fractional part
            int best = 0;
            long bestFrac = -1;
            for (int i = 0; i &lt; 4; i++)
            {
                long frac = (long)v[i] * 100 - (long)scaled[i] * sum;
                if (frac &gt; bestFrac) { bestFrac = frac; best = i; }
            }
            scaled[best]++;
            left--;
        }
        var n = new ScheduleWeights { study = scaled[0], play = scaled[1], club = scaled[2], rest = scaled[3] };
        return n;
    }
    static int Clamp(int v) =&gt; v &lt; 0 ? 0 : (v &gt; 100 ? 100 : v);
}
/// &lt;summary&gt;Five student stats: intelligence, vitality, charm, mood and
/// pocket money. The four core stats live in 0-100 segments; the pocket
/// face accumulates (cap 99999) as the economy face.&lt;/summary&gt;
public struct Stats5
{
    public int intell;
    public int vit;
    public int charm;
    public int mood;
```

```
    public int pocket;
    /// &lt;summary&gt;Clamp law: core stats fold into their segments, the
    /// pocket face accumulates under its cap. Never throws.&lt;/summary&gt;
    public static Stats5 Clamped(Stats5 s)
    {
        s.intell = ClampCore(s.intell);
        s.vit = ClampCore(s.vit);
        s.charm = ClampCore(s.charm);
        s.mood = ClampCore(s.mood);
        s.pocket = s.pocket &lt; 0 ? 0 : (s.pocket &gt; CampusData.PocketCap ? CampusData.PocketCap : s.pocket);
        return s;
    }
    /// &lt;summary&gt;Applies a signed delta then clamps (event effects are
    /// small corrections - the apply face always lands inside the
    /// segments).&lt;/summary&gt;
    public Stats5 Apply(StatDelta fx)
    {
        var s = this;
        s.intell += fx.intell;
        s.vit += fx.vit;
        s.charm += fx.charm;
        s.mood += fx.mood;
        s.pocket += fx.pocket;
        return Clamped(s);
    }
    static int ClampCore(int v) =&gt; v &lt; CampusData.CoreStatMin ? CampusData.CoreStatMin
        : (v &gt; CampusData.CoreStatMax ? CampusData.CoreStatMax : v);
}
/// &lt;summary&gt;A signed five-stat delta (event effects and per-day growth
/// both ride this face).&lt;/summary&gt;
public struct StatDelta
{
    public int intell;
    public int vit;
    public int charm;
    public int mood;
    public int pocket;
    public static StatDelta Zero() =&gt; new StatDelta();
    public int MaxAbs()
    {
        int m = Abs(intell);
        if (Abs(vit) &gt; m) m = Abs(vit);
        if (Abs(charm) &gt; m) m = Abs(charm);
        if (Abs(mood) &gt; m) m = Abs(mood);
        if (Abs(pocket) &gt; m) m = Abs(pocket);
        return m;
    }
    /// &lt;summary&gt;Scales by a weight share (0-100 percent, linear).&lt;/summary&gt;
    public StatDelta Scale(int percentHundredths)
    {
```

```
        var d = this;
        d.intell = d.intell * percentHundredths / 10000;
        d.vit = d.vit * percentHundredths / 10000;
        d.charm = d.charm * percentHundredths / 10000;
        d.mood = d.mood * percentHundredths / 10000;
        d.pocket = d.pocket * percentHundredths / 10000;
        return d;
    }
    static int Abs(int v) =&gt; v &lt; 0 ? -v : v;
}
/// &lt;summary&gt;One school-stage promotion gate: the dual-track law requires
/// the numeric thresholds AND the flag event together (a stage is a new
/// gameplay ticket, never a wall).&lt;/summary&gt;
public sealed class StageGateDef
{
    public int stage;          // the gate's stage (1-based)
    public int nextStage;      // the promoted stage
    public int minIntell;
    public int minVit;
    public int minCharm;
    public string flagEventId = "";
}
/// &lt;summary&gt;One ending line resolution profile (the graduate face).&lt;/summary&gt;
public sealed class EndingDef
{
    public string id = "";
    public int minIntell;
    public int minVit;
    public int minCharm;
}
/// &lt;summary&gt;One campus event card: stage-scoped, typed, small five-stat
/// corrections and an optional goodwill/chain descriptor. Pure data.&lt;/summary&gt;
public sealed class EventDef
{
    public string id = "";
    public int stage;          // 1-5 scope, 0 = any stage
    public string type = "";   // comedy / crisis / social / positive
    public StatDelta fx;       // base effect (choice-side uplifts are the engine face)
    public string chain = "";  // chain descriptor, "" = none
    public int affinity;       // goodwill bump the event may book (0 = none)
}
/// &lt;summary&gt;One exam row on the school-year calendar.&lt;/summary&gt;
public sealed class ExamRow
{
    public int stage;          // the school year this row lives in (1-based)
    public string type = "";   // midterm / final
    public int atDay;          // game-day the exam lands on
    public bool promotion;     // stage-final promotion exams only
    public string eventId = "";
}
```

```
public static class CampusData
{
    public const int SchemaVersion = 2;   // v2: per-stage promotion fold law (SP03 digest sec 1)
    public const int StatCount = 5;
    public const int CoreStatMin = 0;
    public const int CoreStatMax = 100;
    public const int PocketCap = 99999;
    public const int WeightSlots = 4;
    public const int WeightSum = 100;
    public const int StageCount = 5;       // kindergarten -&gt; graduate
    public const int EventCount = 10;      // S1 starter table E01-E10
    public const int EventFxMaxAbs = 5;    // event effects stay small corrections
    public const int EventChainCount = 2;  // E04 truce + E10 three-step reversal
    public const int EventAffinityCount = 2; // E04 truce + E09 umbrella share
    /// &lt;summary&gt;Stage names (English telemetry face).&lt;/summary&gt;
    public static readonly string[] StageNames =
        { "KINDERGARTEN", "PRIMARY", "MIDDLE SCHOOL", "HIGH SCHOOL", "GRADUATE" };
    /// &lt;summary&gt;School-year length per stage in game-days.&lt;/summary&gt;
    public static readonly int[] YearDays = { 60, 90, 90, 90, 45 };
    // ---- growth rates: per game-day at a 100% weight share (linear by
    // share; the T005 engine scales with Scale). The trade faces:
    // study grinds intelligence and burns mood, play lifts mood/charm
    // and foregoes study progress (the int face), club builds charm at a
    // vitality cost, rest restores vitality and mood. ----
    public static readonly StatDelta StudyRate = new StatDelta { intell = 4, mood = -4 };
    public static readonly StatDelta PlayRate = new StatDelta { intell = -2, charm = 6, mood = 10 };
    public static readonly StatDelta ClubRate = new StatDelta { charm = 8, vit = -2 };
    public static readonly StatDelta RestRate = new StatDelta { vit = 8, mood = 3 };
    /// &lt;summary&gt;Weekly pocket allowance per stage (the economy face).&lt;/summary&gt;
    public static readonly int[] WeeklyAllowance = { 0, 5, 10, 15, 15 };
    // ---- comedy tension lines (the two-way burnout poles) ----
    public const int MoodCrashLine = 15;    // all-study crash family arms at/below
    public const int ExamFailLine = 40;     // flunk family arms below
    // ---- pacing anchors (north-star windows) ----
    public const int P1FirstDaySeconds = 60;    // create + first-day stat beats
    public const int P50FirstExamSeconds = 300; // first midterm + first event burst
    // ---- offline progression (SYS07) ----
    public const int OfflineRatePercent = 50;   // offline advances at half the online pace
    public const int OfflineCapHours = 12;     // per-absence cap
    // ---- per-stage promotion fold (schema v2, SP03 digest sec 1) ----
    public const int StageFoldStatPercent = 20;  // core stats fold at every successful promotion (the per-stage re-
    // ---- repeat-year spiral (the senior-year slice, matrix #45 star-path
    // loss beat): a missed promotion year erodes momentum and morale - the
    // repeat must sting so the gate stays a real decision, and the
    // looping student drifts further from the front of the class. ----
    public const int RepeatSpiralIntell = -8;
    public const int RepeatSpiralMood = -8;
    // ---- second-run inheritance (M08) ----
    public const int InheritStatPercent = 20;  // stat carry into the next journey
    public const bool InheritAlbumKept = true;  // the growth album persists
```

```
// (G08-T005 slice B). Builds its whole canvas in code (no scene authoring
// yet): hook caption, one-tap gender cards, four preset weight-disc cards and
// the live five-stat readout. Faces follow the X027 art-first law - when real
// art ships in S2.5 under the CampusPlaceholder stems it wins automatically.
// Every UI sentence routes through CampusTexts keys (zero scattered copy);
// the sfx channel is event-driven (the growth_sfx beat); the audio face
// subscribes the same bus through CampusAudio (B642 prep - stems stay
// silent until the cloudF wave lands, then the catalog lights them up).
using System.Collections.Generic;
using Sy.Core;
using Sy.Social;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.UI;
namespace Sy.CrazyCampus
{
    public sealed class CampusStageView : MonoBehaviour
    {
        CampusStage _stage;
        Transform _canvas;
        Text _caption;
        Text _stats;
        Text _day;
        // popup discipline (U184-B P1): the full-screen dim mask under the
        // three show cards - darkens the play face and eats stray taps
        Image _dimMask;
        Image _student;
        Image _classroom;
        readonly Image[] _genderCards = new Image[2];
        readonly Image[] _discCards = new Image[CampusStage.Presets.Length];
        readonly List&lt;CampusStageEvent&gt; _pending = new List&lt;CampusStageEvent&gt;();
        Image _eventFrame;
        Image _eventArt;
        Text _eventTitle;
        Text _eventStory;
        Text _eventFx;
        Text _eventHint;
        Text _eventToast;
        int _drained;
        float _pulseLeft;
        float _eventCardLeft;
        float _toastLeft;
        // exam show layer (SYS03, T007 slice B): the result overlay card and
        // the pop grade board plate under the stat readout
        Image _examFrame;
        Text _examTitle;
        Text _examScore;
        Text _examVerdict;
        Text _examComedy;
        Text _examHint;
```

```
float _examCardLeft;
Text _boardGrades;
// stage show layer (SYS05, T008 slice B): the Crazy Channel MC
// ceremony overlay, the ending review overlay with the
// second-journey confirm flow, and the growth-album plate
Image _mcFrame;
Text _mcBanner;
Text _mcMain;
Text _mcHint;
float _mcCardLeft;
Image _endFrame;
Text _endTitle;
Text _endMatch;
Text _endCarry;
Text _endAlbum;
Text _endHint;
Text _endButtonText;
bool _endArmed;
Image _albumButton;
Image _albumFrame;
Text _albumLines;
Text _albumHint;
bool _albumOpen;
// about face (T036): the AIGC-label duty plate - bottom-bar entry
// beside the album button, the panel mirrors the album frame law
Image _aboutButton;
Image _aboutFrame;
Text _aboutAigc;
Text _aboutOdds;
Text _aboutAdcap;
Text _aboutAge;
Text _aboutCloseHint;
bool _aboutOpen;
// share face (SYS08, T010): the album / ending share buttons and the
// card compose service (a settle-moment stall point, never a battle frame)
Text _endShareText;
Text _albumShareText;
// show-signal banner (T023): the big accent beat over the whole
// show - plays once per fresh signal kind while playing (the
// play-mode sequencer owns the pop, the shared timer owns the
// edit-mode patrol face)
Text _signalBanner;
float _signalBannerLeft;
readonly HashSet&lt;string&gt; _signalsShown = new HashSet&lt;string&gt;();
UnityEngine.Coroutine _signalShow;
// codex tabs (T010 slice C): the three-book face - classmate codex,
// event codex, graduation album. The tab bodies read straight off the
// codex / album data faces (zero view re-deriving).
readonly Image[] _codexTabs = new Image[3];
readonly Text[] _codexTabLabels = new Text[3];
```

```
int _codexTab;   // 0=classmates 1=events 2=grad photos
// wiring slice B (T021 r2): the classmates-tab portrait rail (ten
// cast seats plus the four-seat faculty row) and the event-card
// actor slots - a discovered seat reads its real face, a locked seat
// keeps the ??? silhouette, the E05 dodgeball overrun beat seats
// the PE teacher beside the athlete.
readonly Image[] _codexPortraits = new Image[10];
readonly Image[] _codexFaculty = new Image[4];
Image _eventActor;
Image _eventTeacher;
const string PeTeacherBeatEventId = "E05";
readonly ShareCardService _shareSvc = new ShareCardService();
void Start()
{
    // G08-T012 boot recovery: read the local-authority save first - a
    // resumed run lands straight back inside the last day face, an
    // empty/corrupt slot starts a honest fresh student; then wire the
    // day-close booking seam so every resolved day persists on the beat.
    _stage = CampusSaveWire.LoadStageOrDefault(CampusSaveWire.ActiveSlot);
    CampusStage.DayCloseBooker = CampusSaveWire.OnDayClosed;
    // G08-T014: the daily LiveOps face rides the same host arming -
    // the device-local day key is the only production source (the
    // ad-cap one-boundary law family, zero server faces).
    CampusLiveOps.DayKeySource = Sy.Platform.AdCapPolicy.TodayKey;
    BuildUi();
    // T023: a resumed run never replays signals that landed before
    // this boot - only fresh beats while playing play the banner.
    foreach (var sig in CampusShowSignals.Derive(_stage)) _signalsShown.Add(sig.kind);
}
void Update()
{
    _stage.Advance(Time.deltaTime);
    DrainEvents();
    if (_pulseLeft &gt; 0f &amp;&amp; _student != null)
    {
        _pulseLeft -= Time.deltaTime;
        float k = Mathf.Max(0f, _pulseLeft / 0.40f);
        _student.color = Color.Lerp(Color.white, CampusPlaceholder.AccentLime, k);
    }
    if (_eventCardLeft &gt; 0f)
    {
        _eventCardLeft -= Time.deltaTime;
        if (_eventCardLeft &lt;= 0f) HideEventCard();
    }
    if (_toastLeft &gt; 0f)
    {
        _toastLeft -= Time.deltaTime;
        if (_toastLeft &lt;= 0f &amp;&amp; _eventToast != null) _eventToast.gameObject.SetActive(false);
    }
    if (_signalBannerLeft &gt; 0f)
```

```
        {
            _signalBannerLeft -= Time.deltaTime;
            if (_signalBannerLeft &lt;= 0f &amp;&amp; _signalBanner != null)
                _signalBanner.gameObject.SetActive(false);
        }
        if (_examCardLeft &gt; 0f)
        {
            _examCardLeft -= Time.deltaTime;
            if (_examCardLeft &lt;= 0f) HideExamCard();
        }
        if (_mcCardLeft &gt; 0f)
        {
            _mcCardLeft -= Time.deltaTime;
            if (_mcCardLeft &lt;= 0f) HideCeremony();
        }
        SyncPopupDim();
    }
    /// &lt;summary&gt;Popup-dim single source (U184-B P1+P2): the mask follows
    /// the three show cards and the three dwell panels (ending / album /
    /// about) live active state every frame - none of the Show / Hide /
    /// Toggle methods ever touch it, so overlapping popups can never
    /// desync a refcount (frame-driven single source). Every panel is
    /// created later in BuildUi than the mask, so it renders above it:
    /// the mask only dims the gameplay face and eats stray taps outside
    /// the panel body (17-seq mask law, modal raycast blocking).&lt;/summary&gt;
    void SyncPopupDim()
    {
        if (_dimMask == null) return;
        bool wantDim = (_eventFrame != null &amp;&amp; _eventFrame.gameObject.activeSelf)
            || (_examFrame != null &amp;&amp; _examFrame.gameObject.activeSelf)
            || (_mcFrame != null &amp;&amp; _mcFrame.gameObject.activeSelf)
            || (_endFrame != null &amp;&amp; _endFrame.gameObject.activeSelf)
            || (_albumFrame != null &amp;&amp; _albumFrame.gameObject.activeSelf)
            || (_aboutFrame != null &amp;&amp; _aboutFrame.gameObject.activeSelf);
        if (_dimMask.gameObject.activeSelf != wantDim)
            _dimMask.gameObject.SetActive(wantDim);
    }
    void BuildUi()
    {
        // Known-infrastructure red line: a UGUI scene must never end up
        // without an EventSystem, so the view self-heals it on boot.
        if (FindObjectOfType&lt;EventSystem&gt;() == null)
        {
            var es = new GameObject("EventSystem", typeof(EventSystem), typeof(StandaloneInputModule));
            // Play-mode only: DontDestroyOnLoad throws in edit mode, and the
            // editor screenshot patrol builds this view outside play mode.
            if (Application.isPlaying) DontDestroyOnLoad(es);
        }
        var canvasGo = new GameObject("CampusCanvas", typeof(Canvas), typeof(CanvasScaler), typeof(GraphicRaycaster)
        var canvas = canvasGo.GetComponent&lt;Canvas&gt;();
```

```
canvas.renderMode = RenderMode.ScreenSpaceOverlay;
var scaler = canvasGo.GetComponent&lt;CanvasScaler&gt;();
scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
scaler.referenceResolution = new Vector2(1080, 1440);
// safe-area container (17 doc section 4, U184-B audit P1): every
// face parents under one full-stretch SafeArea child so
// Screen.safeArea (notch, corner radius) squeezes the play field
// instead of cropping must-see faces. Where safeArea equals the
// full screen (editor gates, batchmode) offsets stay zero, so
// the frozen layout and every pixel probe keep their anchors.
var safeGo = new GameObject("SafeArea", typeof(RectTransform));
safeGo.transform.SetParent(canvasGo.transform, false);
var safeRt = (RectTransform)safeGo.transform;
safeRt.anchorMin = Vector2.zero;
safeRt.anchorMax = Vector2.one;
safeRt.offsetMin = safeRt.offsetMax = Vector2.zero;
safeGo.AddComponent&lt;Sy.Core.SafeAreaFitter&gt;();
_canvas = safeGo.transform;
_caption = MakeText(_canvas, "CaptionText", CampusTexts.Text(CampusTexts.KeyHookCaption),
    new Vector2(0, CampusLayout.HookCaptionY), 52, CampusPlaceholder.AccentLime);
_caption.gameObject.SetActive(false); // fires with the hook beat
// show-signal banner (T023): the three hook-window signals play
// their perceivable face big - a pop banner over the classroom.
_signalBanner = MakeText(_canvas, "SignalBanner", "",
    new Vector2(0, CampusLayout.SignalBannerY), 44, CampusPlaceholder.AccentLime);
_signalBanner.rectTransform.sizeDelta = new Vector2(1000, 64);
_signalBanner.raycastTarget = false;
_signalBanner.gameObject.SetActive(false);
_classroom = MakeBlock(_canvas, "ClassroomFace", CampusPlaceholder.Classroom(1),
    new Vector2(0, -330), 380);
_student = MakeBlock(_canvas, "StudentFace", CampusPlaceholder.StudentMale(),
    new Vector2(0, -330), 140);
_student.gameObject.SetActive(false); // shows on the creation beat
for (int i = 0; i &lt; _genderCards.Length; i++)
{
    int gender = i + 1; // 1 boy / 2 girl
    var card = MakeBlock(_canvas, "GenderCard" + i,
        CampusPlaceholder.Face(gender == 1 ? CampusPlaceholder.StudentMStem : CampusPlaceholder.StudentFStem
            gender == 1 ? CampusPlaceholder.AccentLime : new Color(0.96f, 0.62f, 0.84f), 32),
        new Vector2((i - 0.5f) * 300f, -640), 130);
    var label = MakeText(card.transform, "Label",
        CampusTexts.Text(gender == 1 ? CampusTexts.KeyGenderBoy : CampusTexts.KeyGenderGirl),
        new Vector2(0, -78), 34, Color.white);
    label.raycastTarget = false;
    var button = card.gameObject.AddComponent&lt;Button&gt;();
    button.targetGraphic = card;
    int captured = gender;
    button.onClick.AddListener(() =&gt; OnGenderTapped(captured));
    _genderCards[i] = card;
}
```

```
for (int i = 0; i &lt; _discCards.Length; i++)
{
    var card = MakeBlock(_canvas, "DiscCard" + i, CampusPlaceholder.Panel(),
        new Vector2((i - 1.5f) * 250f, -1000), 220);
    var w = CampusStage.Presets[i];
    var label = MakeText(card.transform, "Shares",
        CampusTexts.Text(CampusTexts.KeyWeightStudy) + " " + w.study + "\n"
        + CampusTexts.Text(CampusTexts.KeyWeightPlay) + " " + w.play + "\n"
        + CampusTexts.Text(CampusTexts.KeyWeightClub) + " " + w.club + "\n"
        + CampusTexts.Text(CampusTexts.KeyWeightRest) + " " + w.rest,
        new Vector2(0, 0), 30, Color.white);
    label.raycastTarget = false;
    var button = card.gameObject.AddComponent&lt;Button&gt;();
    button.targetGraphic = card;
    int captured = i;
    button.onClick.AddListener(() =&gt; OnDiscTapped(captured));
    _discCards[i] = card;
}
_stats = MakeText(_canvas, "StatsText", StatsLine(_stage.Engine.Stats),
    new Vector2(0, -1180), 38, Color.white);
_day = MakeText(_canvas, "DayText", "DAY 0",
    new Vector2(0, -1120), 30, CampusPlaceholder.AccentLime);
// popup discipline (U184-B P1): the dim mask under the card
// family - full-screen black at 50%, raycastTarget=true so a
// stray tap outside an open card dies on the mask instead of
// the play face. Built before the card family so the cards
// paint above it; hidden until a card opens (17h mask law).
var dimGo = new GameObject("PopupDimMask", typeof(Image));
dimGo.transform.SetParent(_canvas, false);
_dimMask = dimGo.GetComponent&lt;Image&gt;();
_dimMask.sprite = null; // solid color face - no sprite, cheap fill
_dimMask.color = new Color(0f, 0f, 0f, 0.50f);
_dimMask.raycastTarget = true;
var dimRt = _dimMask.rectTransform;
dimRt.anchorMin = Vector2.zero;
dimRt.anchorMax = Vector2.one;
dimRt.offsetMin = dimRt.offsetMax = Vector2.zero;
_dimMask.gameObject.SetActive(false);
// Event card (SYS04 face, T006 slice B): the series event-card
// format above the bottom readout - pixel show placeholder,
// one-line story, stat fx row, tap-or-3s dismiss. Hidden until
// the first card beat arrives on the bus.
_eventFrame = MakeBlock(_canvas, "EventCardFrame", CampusPlaceholder.Panel(),
    new Vector2(0, -800), 940);
// the panel block: size to an exact 940x150 card face
ScaleTo(_eventFrame.rectTransform, 940f, 150f);
_eventArt = MakeBlock(_canvas, "EventCardArt", CampusPlaceholder.EventCard(),
    new Vector2(-330, -800), 110);
_eventArt.raycastTarget = false;
// wiring slice B: the actor rail on the card's left edge - the
```

```
// classmate face of the mapped cast seat, and the PE teacher on
// the dodgeball overrun beat. Both start hidden with the card.
_eventActor = MakeBlock(_canvas, "EventCardActor", CampusPlaceholder.LockedFace(),
    new Vector2(-430, -762), 72);
_eventActor.raycastTarget = false;
_eventActor.gameObject.SetActive(false);
_eventTeacher = MakeBlock(_canvas, "EventCardTeacher", CampusPlaceholder.TeacherFace("pe"),
    new Vector2(-430, -838), 72);
_eventTeacher.raycastTarget = false;
_eventTeacher.gameObject.SetActive(false);
_eventTitle = MakeText(_canvas, "EventCardTitle",
    CampusTexts.Text(CampusTexts.KeyEventCardFrame),
    new Vector2(90, -742), 28, CampusPlaceholder.AccentLime);
_eventTitle.raycastTarget = false;
_eventStory = MakeText(_canvas, "EventCardStory", "",
    new Vector2(90, -788), 26, Color.white);
_eventStory.rectTransform.sizeDelta = new Vector2(680, 90);
_eventStory.raycastTarget = false;
_eventFx = MakeText(_canvas, "EventCardFx", "",
    new Vector2(90, -850), 24, CampusPlaceholder.AccentLime);
_eventFx.raycastTarget = false;
_eventHint = MakeText(_canvas, "EventCardHint",
    CampusTexts.Text(CampusTexts.KeyEventCardDismiss),
    new Vector2(0, -864), 18, new Color(0.70f, 0.72f, 0.75f, 0.90f));
_eventHint.raycastTarget = false;
var dismiss = _eventFrame.gameObject.AddComponent&lt;Button&gt;();
dismiss.targetGraphic = _eventFrame;
dismiss.onClick.AddListener(HideEventCard);
_eventToast = MakeText(_canvas, "EventToast", "",
    new Vector2(0, -690), 30, CampusPlaceholder.AccentLime);
_eventToast.raycastTarget = false;
HideEventCard();
_eventToast.gameObject.SetActive(false);
// Exam result card (SYS03 face, T007 slice B): the exam show
// overlay lives in the same classroom scene (S08 law) - pixel
// placeholder frame, the paper title, the big score window, the
// verdict, the guess-C / repeat-year comedy lines, 4s auto-hide
// or one tap. Hidden until the first exam beat rides the bus.
_examFrame = MakeBlock(_canvas, "ExamCardFrame", CampusPlaceholder.Panel(),
    new Vector2(0, -620), 940);
ScaleTo(_examFrame.rectTransform, 940f, 320f);
_examTitle = MakeText(_canvas, "ExamCardTitle", "",
    new Vector2(0, -495), 34, CampusPlaceholder.AccentLime);
_examTitle.raycastTarget = false;
_examScore = MakeText(_canvas, "ExamCardScore", "",
    new Vector2(0, -585), 56, Color.white);
_examScore.raycastTarget = false;
_examVerdict = MakeText(_canvas, "ExamCardVerdict", "",
    new Vector2(0, -672), 30, CampusPlaceholder.AccentLime);
_examVerdict.raycastTarget = false;
```

```
_examComedy = MakeText(_canvas, "ExamCardComedy", "",
    new Vector2(0, -715), 24, new Color(0.96f, 0.62f, 0.84f));
_examComedy.raycastTarget = false;
_examHint = MakeText(_canvas, "ExamCardHint",
    CampusTexts.Text(CampusTexts.KeyEventCardDismiss),
    new Vector2(0, -757), 18, new Color(0.70f, 0.72f, 0.75f, 0.90f));
_examHint.raycastTarget = false;
var examDismiss = _examFrame.gameObject.AddComponent&lt;Button&gt;();
examDismiss.targetGraphic = _examFrame;
examDismiss.onClick.AddListener(HideExamCard);
HideExamCard();
// Pop grade board (the report-card plate, T007 slice B): under
// the stat readout - one token per resolved exam,
// refreshed straight off the exam engine history (BoardLine is
// the shared data-face law, zero view re-deriving). Real art
// (g08_ui_scoreboard) wins automatically when it ships (X027).
ScaleTo(MakeBlock(_canvas, "ReportCardPlate", CampusPlaceholder.Panel(),
    new Vector2(0, -1300), 700).rectTransform, 700f, 90f);
MakeText(_canvas, "ReportCardTitle",
    CampusTexts.Text(CampusTexts.KeyReportCard),
    new Vector2(0, -1272), 22, CampusPlaceholder.AccentLime).raycastTarget = false;
_boardGrades = MakeText(_canvas, "ReportCardGrades", "",
    new Vector2(0, -1320), 26, Color.white);
_boardGrades.raycastTarget = false;
RefreshBoard();
// Stage show layer (SYS05, T008 slice B): the Crazy Channel MC
// ceremony overlay - the news banner, the stage-up headline (or
// the repeat / graduate-segment comedy lines), 4s auto-hide or
// one tap. Hidden until the first stage beat rides the bus.
_mcFrame = MakeBlock(_canvas, "StageCeremonyFrame", CampusPlaceholder.Panel(),
    new Vector2(0, -430), 940);
ScaleTo(_mcFrame.rectTransform, 940f, 240f);
_mcBanner = MakeText(_canvas, "StageCeremonyBanner",
    CampusTexts.Text(CampusTexts.KeyStageUpMc),
    new Vector2(0, -350), 30, CampusPlaceholder.AccentLime);
_mcBanner.raycastTarget = false;
_mcMain = MakeText(_canvas, "StageCeremonyMain", "",
    new Vector2(0, -440), 40, Color.white);
_mcMain.raycastTarget = false;
_mcHint = MakeText(_canvas, "StageCeremonyHint",
    CampusTexts.Text(CampusTexts.KeyEventCardDismiss),
    new Vector2(0, -528), 18, new Color(0.70f, 0.72f, 0.75f, 0.90f));
_mcHint.raycastTarget = false;
var mcDismiss = _mcFrame.gameObject.AddComponent&lt;Button&gt;();
mcDismiss.targetGraphic = _mcFrame;
mcDismiss.onClick.AddListener(HideCeremony);
HideCeremony();
// Ending review overlay (the graduation face, T008 slice B):
// persistent card - the ending line title, the matched /
// fallback verdict, the next-kid carry, the album long-image
```

```
// review and the two-tap second-journey confirm flow. No
// auto-hide: the moment owns the screen until the player
// decides.
_endFrame = MakeBlock(_canvas, "EndingFrame", CampusPlaceholder.Panel(),
    new Vector2(0, -700), 940);
ScaleTo(_endFrame.rectTransform, 940f, 760f);
_endTitle = MakeText(_canvas, "EndingTitle", "",
    new Vector2(0, -360), 44, CampusPlaceholder.AccentLime);
_endTitle.raycastTarget = false;
_endMatch = MakeText(_canvas, "EndingMatch", "",
    new Vector2(0, -430), 24, Color.white);
_endMatch.raycastTarget = false;
_endCarry = MakeText(_canvas, "EndingCarry", "",
    new Vector2(0, -485), 24, new Color(0.96f, 0.62f, 0.84f));
_endCarry.raycastTarget = false;
_endAlbum = MakeText(_canvas, "EndingAlbum",
    CampusTexts.Text(CampusTexts.KeyAlbumTitle),
    new Vector2(0, -640), 26, Color.white);
_endAlbum.rectTransform.sizeDelta = new Vector2(860, 300);
_endAlbum.raycastTarget = false;
_endHint = MakeText(_canvas, "EndingHint",
    CampusTexts.Text(CampusTexts.KeyAlbumEmpty),
    new Vector2(0, -850), 18, new Color(0.70f, 0.72f, 0.75f, 0.90f));
_endHint.raycastTarget = false;
var endGo = new GameObject("EndingNewJourneyButton", typeof(Image), typeof(Button));
endGo.transform.SetParent(_canvas, false);
var endBtnImg = endGo.GetComponent&lt;Image&gt;();
endBtnImg.sprite = CampusPlaceholder.Panel();
endBtnImg.SetNativeSize();
var endBtnRt = endBtnImg.rectTransform;
endBtnRt.anchorMin = endBtnRt.anchorMax = new Vector2(0.5f, 1f);
endBtnRt.anchoredPosition = new Vector2(0, -960);
ScaleTo(endBtnRt, 640f, 96f);
_endButtonText = MakeText(endGo.transform, "EndingNewJourneyLabel",
    CampusTexts.Text(CampusTexts.KeyNewJourney),
    new Vector2(0, 0), 26, CampusPlaceholder.AccentLime);
_endButtonText.raycastTarget = false;
var endButton = endGo.GetComponent&lt;Button&gt;();
endButton.targetGraphic = endBtnImg;
endButton.onClick.AddListener(OnNewJourneyTapped);
// share button (T010): the ending moment owns the brag - the
// graduation context is armed by construction here.
var endShareGo = new GameObject("EndingShareButton", typeof(Image), typeof(Button));
endShareGo.transform.SetParent(_canvas, false);
var endShareImg = endShareGo.GetComponent&lt;Image&gt;();
endShareImg.sprite = CampusPlaceholder.Panel();
endShareImg.SetNativeSize();
var endShareRt = endShareImg.rectTransform;
endShareRt.anchorMin = endShareRt.anchorMax = new Vector2(0.5f, 1f);
endShareRt.anchoredPosition = new Vector2(0, -1046);
```

```
ScaleTo(endShareRt, 480f, 64f);
_endShareText = MakeText(endShareGo.transform, "EndingShareLabel",
    CampusTexts.Text(CampusTexts.KeyAlbumShare),
    new Vector2(0, 0), 24, CampusPlaceholder.AccentLime);
_endShareText.raycastTarget = false;
var endShareButton = endShareGo.GetComponent&lt;Button&gt;();
endShareButton.targetGraphic = endShareImg;
endShareButton.onClick.AddListener(OnShareTapped);
HideEnding();
// Growth album plate (T008 slice B): the bottom-bar album
// button opens the long-image review panel - entries read
// straight off the album data face (zero view re-deriving).
_albumButton = MakeBlock(_canvas, "AlbumButton", CampusPlaceholder.Panel(),
    new Vector2(0, -1400), 480);
ScaleTo(_albumButton.rectTransform, 480f, 64f);
MakeText(_canvas, "AlbumButtonLabel",
    CampusTexts.Text(CampusTexts.KeyAlbumTitle),
    new Vector2(0, -1400), 24, CampusPlaceholder.AccentLime).raycastTarget = false;
var albumToggle = _albumButton.gameObject.AddComponent&lt;Button&gt;();
albumToggle.targetGraphic = _albumButton;
albumToggle.onClick.AddListener(ToggleAlbum);
_albumFrame = MakeBlock(_canvas, "AlbumPanel", CampusPlaceholder.Panel(),
    new Vector2(0, -700), 900);
ScaleTo(_albumFrame.rectTransform, 900f, 820f);
MakeText(_canvas, "AlbumPanelTitle",
    CampusTexts.Text(CampusTexts.KeyAlbumTitle),
    new Vector2(0, -340), 36, CampusPlaceholder.AccentLime).raycastTarget = false;
_albumLines = MakeText(_canvas, "AlbumPanelLines", "",
    new Vector2(0, -660), 26, Color.white);
_albumLines.rectTransform.sizeDelta = new Vector2(820, 340);
_albumLines.raycastTarget = false;
_albumHint = MakeText(_canvas, "AlbumPanelHint",
    CampusTexts.Text(CampusTexts.KeyEventCardDismiss),
    new Vector2(0, -1055), 18, new Color(0.70f, 0.72f, 0.75f, 0.90f));
_albumHint.raycastTarget = false;
var albumDismiss = _albumFrame.gameObject.AddComponent&lt;Button&gt;();
albumDismiss.targetGraphic = _albumFrame;
albumDismiss.onClick.AddListener(HideAlbum);
// share button (T010): the persistent brag entry - before the
// first graduation it answers with the context-less guard line.
var albumShareGo = new GameObject("AlbumShareButton", typeof(Image), typeof(Button));
albumShareGo.transform.SetParent(_canvas, false);
var albumShareImg = albumShareGo.GetComponent&lt;Image&gt;();
albumShareImg.sprite = CampusPlaceholder.Panel();
albumShareImg.SetNativeSize();
var albumShareRt = albumShareImg.rectTransform;
albumShareRt.anchorMin = albumShareRt.anchorMax = new Vector2(0.5f, 1f);
albumShareRt.anchoredPosition = new Vector2(0, -930);
ScaleTo(albumShareRt, 480f, 64f);
_albumShareText = MakeText(albumShareGo.transform, "AlbumShareLabel",
```

```
    CampusTexts.Text(CampusTexts.KeyAlbumShare),
    new Vector2(0, 0), 24, CampusPlaceholder.AccentLime);
_albumShareText.raycastTarget = false;
var albumShareButton = albumShareGo.GetComponent&lt;Button&gt;();
albumShareButton.targetGraphic = albumShareImg;
albumShareButton.onClick.AddListener(OnShareTapped);
// three-book tabs (T010 slice C): CLASSMATES / EVENTS / GRAD
// PHOTOS - the tab bodies render the codex / album data faces
// verbatim; the active tab reads accent, the idle tabs dim.
for (int i = 0; i &lt; _codexTabs.Length; i++)
{
    int idx = i;
    _codexTabs[i] = MakeBlock(_canvas, "CodexTab" + i, CampusPlaceholder.Panel(),
        new Vector2(-300 + i * 300, -430), 64);
    ScaleTo(_codexTabs[i].rectTransform, 260f, 56f);
    _codexTabLabels[i] = MakeText(_canvas, "CodexTabLabel" + i, CodexTabLabel(i),
        new Vector2(-300 + i * 300, -430), 20, CampusPlaceholder.AccentLime);
    _codexTabLabels[i].raycastTarget = false;
    var tabButton = _codexTabs[i].gameObject.AddComponent&lt;Button&gt;();
    tabButton.targetGraphic = _codexTabs[i];
    tabButton.onClick.AddListener(() =&gt; OnCodexTabTapped(idx));
}
// wiring slice B: the classmates-tab portrait rail - ten cast
// seats in frozen order plus the four-seat faculty row. The rail
// hides with the album; RefreshAlbum swaps discovered seats to
// their real faces (locked seats keep the silhouette).
for (int i = 0; i &lt; _codexPortraits.Length; i++)
{
    _codexPortraits[i] = MakeBlock(_canvas, "CodexPortrait" + i,
        CampusPlaceholder.LockedFace(), new Vector2(-369 + i * 82, -516), 72);
    _codexPortraits[i].raycastTarget = false;
    _codexPortraits[i].gameObject.SetActive(false);
}
for (int i = 0; i &lt; _codexFaculty.Length; i++)
{
    _codexFaculty[i] = MakeBlock(_canvas, "CodexFaculty" + i,
        CampusPlaceholder.TeacherFace(CampusAffinity.TeacherStems[i]),
        new Vector2(-123 + i * 82, -600), 72);
    _codexFaculty[i].raycastTarget = false;
    _codexFaculty[i].gameObject.SetActive(false);
}
HideAlbum();
// About face (T036): the AIGC-label duty plate - a bottom-bar
// entry beside the album button (album owns x[-240,240], this
// one sits left, zero overlap) and the disclosure panel mirrors
// the album frame law. All copy rides CampusTexts keys through
// the CampusAbout composer (live tables, zero hardcodes).
_aboutButton = MakeBlock(_canvas, "AboutButton", CampusPlaceholder.Panel(),
    new Vector2(-370, -1400), 200);
ScaleTo(_aboutButton.rectTransform, 200f, 64f);
```

```
        MakeText(_canvas, "AboutButtonLabel",
            CampusTexts.Text(CampusTexts.KeyAboutButton),
            new Vector2(-370, -1400), 20, CampusPlaceholder.AccentLime).raycastTarget = false;
        var aboutToggle = _aboutButton.gameObject.AddComponent&lt;Button&gt;();
        aboutToggle.targetGraphic = _aboutButton;
        aboutToggle.onClick.AddListener(ToggleAbout);
        _aboutFrame = MakeBlock(_canvas, "AboutPanel", CampusPlaceholder.Panel(),
            new Vector2(0, -700), 900);
        ScaleTo(_aboutFrame.rectTransform, 900f, 820f);
        MakeText(_canvas, "AboutPanelTitle",
            CampusTexts.Text(CampusTexts.KeyAboutTitle),
            new Vector2(0, -360), 40, CampusPlaceholder.AccentLime).raycastTarget = false;
        _aboutAigc = MakeText(_canvas, "AboutAigcLine", CampusAbout.AigcLine(),
            new Vector2(0, -470), 24, Color.white);
        _aboutAigc.rectTransform.sizeDelta = new Vector2(820, 70);
        _aboutAigc.raycastTarget = false;
        _aboutOdds = MakeText(_canvas, "AboutOddsLine", CampusAbout.OddsLine(),
            new Vector2(0, -650), 21, Color.white);
        _aboutOdds.rectTransform.sizeDelta = new Vector2(820, 170);
        _aboutOdds.raycastTarget = false;
        _aboutAdcap = MakeText(_canvas, "AboutAdcapLine", CampusAbout.AdCapLine(),
            new Vector2(0, -880), 21, Color.white);
        _aboutAdcap.rectTransform.sizeDelta = new Vector2(820, 120);
        _aboutAdcap.raycastTarget = false;
        _aboutAge = MakeText(_canvas, "AboutAgeLine", CampusAbout.AgeLine(),
            new Vector2(0, -1010), 21, Color.white);
        _aboutAge.raycastTarget = false;
        _aboutCloseHint = MakeText(_canvas, "AboutCloseHint",
            CampusTexts.Text(CampusTexts.KeyAboutClose),
            new Vector2(0, -1060), 18, new Color(0.70f, 0.72f, 0.75f, 0.90f));
        _aboutCloseHint.raycastTarget = false;
        var aboutDismiss = _aboutFrame.gameObject.AddComponent&lt;Button&gt;();
        aboutDismiss.targetGraphic = _aboutFrame;
        aboutDismiss.onClick.AddListener(HideAbout);
        HideAbout();
    }
    void OnGenderTapped(int gender)
    {
        _stage.PickGender(gender);
        DrainEvents();
    }
    void OnDiscTapped(int index)
    {
        _stage.PickPreset(index);
        DrainEvents();
    }
    void DrainEvents()
    {
        _pending.Clear();
        // fresh events only - the stage never rewrites history
```

```
for (int i = _drained; i &lt; _stage.Events.Count; i++) _pending.Add(_stage.Events[i]);
_drained = _stage.Events.Count;
foreach (var e in _pending)
{
    switch (e.type)
    {
        case "evt_campus_hook":
            if (_caption != null) _caption.gameObject.SetActive(true);
            break;
        case "evt_campus_created":
            if (_student != null)
            {
                _student.sprite = _stage.Gender == 1
                    ? CampusPlaceholder.StudentMale()
                    : CampusPlaceholder.StudentFemale();
                _student.gameObject.SetActive(true);
            }
            for (int i = 0; i &lt; _genderCards.Length; i++)
                if (_genderCards[i] != null) _genderCards[i].gameObject.SetActive(false);
            break;
        case "evt_campus_first_plan":
            for (int i = 0; i &lt; _discCards.Length; i++)
                if (_discCards[i] != null)
                    _discCards[i].color = i == _stage.PresetIndex
                        ? CampusPlaceholder.AccentLime
                        : Color.white;
            break;
        case "evt_campus_first_growth":
            // same-tick feedback (R3 face): the first growth beat
            // fires its stem with the number pop - silence until
            // the cloudF wave lands (null-safe law).
            CampusAudio.PlayOneShot(CampusAudio.SfxFirstGrowth);
            if (_stats != null) _stats.text = StatsLine(_stage.Engine.Stats);
            if (_day != null) _day.text = string.Format(CampusTexts.Text(CampusTexts.KeyDayHud), _stage.Engi
            break;
        case "growth_number":
            if (_stats != null) _stats.text = StatsLine(_stage.Engine.Stats);
            if (_day != null) _day.text = string.Format(CampusTexts.Text(CampusTexts.KeyDayHud), _stage.Engi
            break;
        case "growth_visual":
            _pulseLeft = 0.40f; // the student flashes lime
            break;
        case "evt_campus_event_card":
            ShowEventCard(e.detail);
            break;
        case "evt_campus_event_chain":
            {
                // chain beat toast: one line, the step key reads
                // straight from the id:step detail
                int cut = e.detail.LastIndexOf(':');
```

```
            if (cut &gt; 0 &amp;&amp; _eventToast != null)
            {
                _eventToast.text = CampusTexts.Text("chain_" + e.detail.Substring(cut + 1));
                _eventToast.gameObject.SetActive(true);
                _toastLeft = 2.5f;
            }
        }
        break;
    case "evt_exam_result":
        // the exam show: card face reads the freshest grade
        // record straight off the engine history; the board
        // plate refreshes the same drive (zero copies)
        ShowExamCard();
        RefreshBoard();
        break;
    case "evt_exam_guess_c":
        if (_examComedy != null)
            _examComedy.text = CampusTexts.Text(CampusTexts.KeyExamGuessC);
        break;
    case "evt_exam_repeat":
        if (_examComedy != null)
        {
            // a deep-flunk promotion paper books both comedy
            // beats - the lines stack on one card
            string line = CampusTexts.Text(CampusTexts.KeyExamRepeat);
            _examComedy.text = _examComedy.text.Length == 0
                ? line : _examComedy.text + "\n" + line;
        }
        break;
    case "evt_stage_up":
        // same-tick feedback (R3 face): the promotion stem
        // rides the ceremony beat - silence until the cloudF
        // wave lands (null-safe law).
        CampusAudio.PlayOneShot(CampusAudio.SfxStageUp);
        // the ceremony face: the classroom tint rides the new
        // stage (the visual progression), the MC overlay
        // headlines the promotion off the re-scoped stage
        if (_classroom != null)
            _classroom.sprite = CampusPlaceholder.Classroom(_stage.Engine.Stage);
        ShowCeremony(string.Format(CampusTexts.Text(CampusTexts.KeyStageUp),
            CampusTexts.Text(CampusTexts.KeyStageNamePrefix + _stage.Engine.Stage)));
        if (_albumOpen) RefreshAlbum();
        break;
    case "evt_stage_repeat":
        ShowCeremony(CampusTexts.Text(CampusTexts.KeyStageRepeatLine));
        break;
    case "evt_graduate_segment":
        ShowCeremony(CampusTexts.Text(CampusTexts.KeyGraduateSegmentLine));
        break;
    case "evt_grad_ending":
```

```
                ShowEnding();
                break;
            case "evt_new_journey":
                // the spine restarted: the tint walks back to
                // kindergarten, the readouts and the album panel
                // re-read the fresh faces
                HideEnding();
                if (_classroom != null)
                    _classroom.sprite = CampusPlaceholder.Classroom(1);
                if (_stats != null) _stats.text = StatsLine(_stage.Engine.Stats);
                if (_day != null) _day.text = string.Format(CampusTexts.Text(CampusTexts.KeyDayHud), _stage.Engi
                RefreshBoard();
                if (_albumOpen) RefreshAlbum();
                // T023: the fresh journey re-arms the signal beats -
                // whatever still derives after the reset seeds the
                // set, so nothing replays a stale face.
                _signalsShown.Clear();
                foreach (var sig in CampusShowSignals.Derive(_stage))
                    _signalsShown.Add(sig.kind);
                break;
            // growth_sfx: the placeholder beat lives on the bus; the
            // audio face subscribes it when the S2.5 audio batch lands.
        }
    }
    RefreshShowSignals();
}
/// &lt;summary&gt;Plays any signal face that landed since the last drain
/// (T023): fresh kinds only - the banner is the big accent beat on
/// top of the existing card / ceremony / ending shows, never a
/// replacement for them.&lt;/summary&gt;
void RefreshShowSignals()
{
    foreach (var sig in CampusShowSignals.Derive(_stage))
    {
        if (_signalsShown.Contains(sig.kind)) continue;
        _signalsShown.Add(sig.kind);
        if (_signalBanner == null) continue;
        _signalBanner.text = sig.face;
        _signalBanner.gameObject.SetActive(true);
        _signalBannerLeft = 2.60f;
        if (Application.isPlaying)
        {
            if (_signalShow != null) StopCoroutine(_signalShow);
            _signalShow = StartCoroutine(SignalShowSequence());
        }
    }
}
/// &lt;summary&gt;The show sequencer (the T023 show coroutine): a quick
/// pop settles the banner into place while the shared timer owns
/// the hold and the hide - the edit-mode patrol face rides the
```

```
/// same timer law (coroutines only run in play mode).&lt;/summary&gt;
System.Collections.IEnumerator SignalShowSequence()
{
    float pop = 0.30f;
    var rt = _signalBanner.rectTransform;
    Vector3 settled = rt.localScale;
    while (pop &gt; 0f)
    {
        pop -= Time.deltaTime;
        float k = Mathf.Max(0f, pop / 0.30f);
        rt.localScale = settled * (1f + 0.18f * k);
        yield return null;
    }
    rt.localScale = settled;
}
/// &lt;summary&gt;Shows the event card: pixel show placeholder, the
/// one-line story from the id key, and the card's stat fx row
/// (read straight off the engine log - zero state copies). 3s
/// auto-hide, or one tap anywhere on the card.&lt;/summary&gt;
void ShowEventCard(string id)
{
    if (_eventFrame == null) return;
    // same-tick feedback (R3 face): the card reveal stem rides the
    // pop - silence until the cloudF wave lands (null-safe law).
    CampusAudio.PlayOneShot(CampusAudio.SfxEventCard);
    _eventStory.text = CampusTexts.Text("event_" + id.ToLower());
    _eventFx.text = FxLine(FxOfCard(id));
    _eventFrame.gameObject.SetActive(true);
    if (_eventArt != null) _eventArt.gameObject.SetActive(true);
    _eventTitle.gameObject.SetActive(true);
    _eventStory.gameObject.SetActive(true);
    _eventFx.gameObject.SetActive(true);
    _eventHint.gameObject.SetActive(true);
    // wiring slice B: the actor face rides the frozen cast mapping
    // (the 8.2 table IS the actor history) - a mapped seat reads its
    // real classmate face, an unmapped id falls to the generic npc
    // face; the E05 overrun seats the PE teacher too.
    if (_eventActor != null)
    {
        string seat = CampusCodex.SeatOfEvent(id);
        _eventActor.sprite = seat.Length &gt; 0
            ? CampusPlaceholder.ClassmateFace(seat)
            : CampusPlaceholder.Npc();
        _eventActor.gameObject.SetActive(true);
    }
    if (_eventTeacher != null)
        _eventTeacher.gameObject.SetActive(id == PeTeacherBeatEventId);
    _eventCardLeft = 3.0f;
}
void HideEventCard()
```

```
{
    if (_eventFrame == null) return;
    _eventFrame.gameObject.SetActive(false);
    if (_eventArt != null) _eventArt.gameObject.SetActive(false);
    _eventTitle.gameObject.SetActive(false);
    _eventStory.gameObject.SetActive(false);
    _eventFx.gameObject.SetActive(false);
    _eventHint.gameObject.SetActive(false);
    if (_eventActor != null) _eventActor.gameObject.SetActive(false);
    if (_eventTeacher != null) _eventTeacher.gameObject.SetActive(false);
    _eventCardLeft = 0f;
}
/// &lt;summary&gt;Shows the exam result card off the freshest grade record:
/// paper title, big score window, verdict - the comedy line starts
/// empty and the guess-C / repeat beats fill it as they drain. 4s
/// auto-hide, or one tap anywhere on the card.&lt;/summary&gt;
void ShowExamCard()
{
    if (_examFrame == null) return;
    var hist = _stage.ExamEngine.History;
    if (hist.Count == 0) return;
    var r = hist[hist.Count - 1];
    _examTitle.text = CampusTexts.Text(
        r.type == "final" ? CampusTexts.KeyExamFinal : CampusTexts.KeyExamMidterm);
    _examScore.text = string.Format(CampusTexts.Text(CampusTexts.KeyExamResult), r.score);
    _examVerdict.text = CampusTexts.Text(
        r.passed ? CampusTexts.KeyExamPassed : CampusTexts.KeyExamFailed);
    _examComedy.text = "";
    _examFrame.gameObject.SetActive(true);
    _examTitle.gameObject.SetActive(true);
    _examScore.gameObject.SetActive(true);
    _examVerdict.gameObject.SetActive(true);
    _examComedy.gameObject.SetActive(true);
    _examHint.gameObject.SetActive(true);
    _examCardLeft = 4.0f;
}
void HideExamCard()
{
    if (_examFrame == null) return;
    _examFrame.gameObject.SetActive(false);
    _examTitle.gameObject.SetActive(false);
    _examScore.gameObject.SetActive(false);
    _examVerdict.gameObject.SetActive(false);
    _examComedy.gameObject.SetActive(false);
    _examHint.gameObject.SetActive(false);
    _examCardLeft = 0f;
}
/// &lt;summary&gt;Shows the Crazy Channel MC ceremony overlay: the news
/// banner plus the headline line (a stage-up promotion, the
/// growing-up-pending repeat comedy or the graduate-segment
```

```
/// bulletin). 4s auto-hide, or one tap anywhere on the card.&lt;/summary&gt;
void ShowCeremony(string headline)
{
    if (_mcFrame == null) return;
    _mcMain.text = headline;
    _mcFrame.gameObject.SetActive(true);
    _mcBanner.gameObject.SetActive(true);
    _mcMain.gameObject.SetActive(true);
    _mcHint.gameObject.SetActive(true);
    _mcCardLeft = 4.0f;
}
void HideCeremony()
{
    if (_mcFrame == null) return;
    _mcFrame.gameObject.SetActive(false);
    _mcBanner.gameObject.SetActive(false);
    _mcMain.gameObject.SetActive(false);
    _mcHint.gameObject.SetActive(false);
    _mcCardLeft = 0f;
}
/// &lt;summary&gt;Shows the ending review overlay off the freshest
/// graduate resolution (the view never re-derives): the ending line
/// title through the shared key map, the matched / fallback
/// verdict, the carry label, the album long-image review and the
/// second-journey button in its unarmed state. Persistent - no
/// auto-hide, the moment owns the screen.&lt;/summary&gt;
void ShowEnding()
{
    if (_endFrame == null) return;
    var grad = _stage.LastGraduation;
    if (grad == null) return;
    _endTitle.text = CampusTexts.Text(
        CampusStageGateEngine.EndingTextKey(grad.endingId));
    _endMatch.text = CampusTexts.Text(
        grad.matched ? CampusTexts.KeyEndingMatched : CampusTexts.KeyEndingFallback);
    _endCarry.text = string.Format(CampusTexts.Text(CampusTexts.KeyCarryLabel),
        CampusStageGateEngine.CarrySig(grad.inherited));
    _endAlbum.text = _stage.Album.Lines();
    _endHint.text = CampusTexts.Text(CampusTexts.KeyAlbumEmpty);
    _endArmed = false;
    _endButtonText.text = CampusTexts.Text(CampusTexts.KeyNewJourney);
    _endFrame.gameObject.SetActive(true);
    _endTitle.gameObject.SetActive(true);
    _endMatch.gameObject.SetActive(true);
    _endCarry.gameObject.SetActive(true);
    _endAlbum.gameObject.SetActive(true);
    _endHint.gameObject.SetActive(true);
    _endButtonText.transform.parent.gameObject.SetActive(true);
    if (_endShareText != null) _endShareText.transform.parent.gameObject.SetActive(true);
}
```

```
void HideEnding()
{
    if (_endFrame == null) return;
    _endFrame.gameObject.SetActive(false);
    _endTitle.gameObject.SetActive(false);
    _endMatch.gameObject.SetActive(false);
    _endCarry.gameObject.SetActive(false);
    _endAlbum.gameObject.SetActive(false);
    _endHint.gameObject.SetActive(false);
    _endButtonText.transform.parent.gameObject.SetActive(false);
    if (_endShareText != null) _endShareText.transform.parent.gameObject.SetActive(false);
    _endArmed = false;
}
/// &lt;summary&gt;The two-tap second-journey confirm flow: the first tap
/// arms the confirmation line, the second executes StartNewJourney
/// (the carry seeds the next kid; the evt_new_journey beat walks the
/// refresh through DrainEvents).&lt;/summary&gt;
void OnNewJourneyTapped()
{
    if (!_endArmed)
    {
        _endArmed = true;
        _endButtonText.text = CampusTexts.Text(CampusTexts.KeyNewJourneyConfirm);
        return;
    }
    _stage.StartNewJourney();
    DrainEvents();
}
/// &lt;summary&gt;Share tap (T010): composes the growth-album show-off
/// card through the settle-moment pipeline and books the hint line
/// from the machine-readable outcome; a successful compose stamps
/// evt_album_share on the stage beat funnel like any stage beat.&lt;/summary&gt;
void OnShareTapped()
{
    var res = CampusShare.ComposeCard(_shareSvc, _stage);
    string hint = res.Ok
        ? CampusTexts.Text(CampusTexts.KeyShareSaved)
        : CampusTexts.Text(CampusTexts.KeyShareNoContext);
    if (_albumOpen &amp;amp;&amp;amp; _albumHint != null) _albumHint.text = hint;
    else if (_endHint != null) _endHint.text = hint;
}
/// &lt;summary&gt;Toggles the three-book codex panel (T010 slice C): the
/// tab bodies read straight off the codex / album data faces (kept
/// across journeys, the memories never reset).&lt;/summary&gt;
void ToggleAlbum()
{
    if (_aboutOpen) HideAbout();
    if (_albumOpen) { HideAlbum(); return; }
    RefreshAlbum();
    _albumFrame.gameObject.SetActive(true);
```

```
        _albumLines.gameObject.SetActive(true);
        _albumHint.gameObject.SetActive(true);
        for (int i = 0; i &lt; _codexTabs.Length; i++)
        {
            _codexTabs[i].gameObject.SetActive(true);
            _codexTabLabels[i].gameObject.SetActive(true);
        }
        if (_albumShareText != null) _albumShareText.transform.parent.gameObject.SetActive(true);
        _albumOpen = true;
    }
    void HideAlbum()
    {
        if (_albumFrame == null) return;
        _albumFrame.gameObject.SetActive(false);
        _albumLines.gameObject.SetActive(false);
        _albumHint.gameObject.SetActive(false);
        for (int i = 0; i &lt; _codexTabs.Length; i++)
        {
            if (_codexTabs[i] == null) continue;
            _codexTabs[i].gameObject.SetActive(false);
            _codexTabLabels[i].gameObject.SetActive(false);
        }
        if (_albumShareText != null) _albumShareText.transform.parent.gameObject.SetActive(false);
        for (int i = 0; i &lt; _codexPortraits.Length; i++)
            if (_codexPortraits[i] != null) _codexPortraits[i].gameObject.SetActive(false);
        for (int i = 0; i &lt; _codexFaculty.Length; i++)
            if (_codexFaculty[i] != null) _codexFaculty[i].gameObject.SetActive(false);
        _albumOpen = false;
    }
    /// &lt;summary&gt;Toggles the About disclosure panel (T036): the album and
    /// the about plate never share the stage - opening one closes the
    /// other; the panel reads the CampusAbout composer face.&lt;/summary&gt;
    void ToggleAbout()
    {
        if (_aboutOpen) { HideAbout(); return; }
        HideAlbum();
        _aboutFrame.gameObject.SetActive(true);
        _aboutAigc.gameObject.SetActive(true);
        _aboutOdds.gameObject.SetActive(true);
        _aboutAdcap.gameObject.SetActive(true);
        _aboutAge.gameObject.SetActive(true);
        _aboutCloseHint.gameObject.SetActive(true);
        _aboutOpen = true;
    }
    void HideAbout()
    {
        if (_aboutFrame == null) return;
        _aboutFrame.gameObject.SetActive(false);
        _aboutAigc.gameObject.SetActive(false);
        _aboutOdds.gameObject.SetActive(false);
```

```
    _aboutAdcap.gameObject.SetActive(false);
    _aboutAge.gameObject.SetActive(false);
    _aboutCloseHint.gameObject.SetActive(false);
    _aboutOpen = false;
}
void OnCodexTabTapped(int index)
{
    _codexTab = index;
    RefreshAlbum();
}
string CodexTabLabel(int index)
{
    if (index == 0) return CampusTexts.Text(CampusTexts.KeyCodexTabClassmates);
    if (index == 1) return CampusTexts.Text(CampusTexts.KeyCodexTabEvents);
    return CampusTexts.Text(CampusTexts.KeyCodexTabPhotos);
}
/// &lt;summary&gt;The tab body law: classmates / events read the derived
/// codex (derive-on-read, zero caching), the photos tab keeps the
/// album long-image face (the slice-B behavior unchanged).&lt;/summary&gt;
string CodexTabBody()
{
    if (_codexTab == 0) return CampusCodex.Derive(_stage.EventEngine).SeatsFace();
    if (_codexTab == 1) return CampusCodex.Derive(_stage.EventEngine).EventsFace();
    var lines = _stage.Album.Lines();
    return lines.Length == 0
        ? CampusTexts.Text(CampusTexts.KeyAlbumEmpty) : lines;
}
void RefreshAlbum()
{
    if (_albumLines == null) return;
    _albumLines.text = CodexTabBody();
    // wiring slice B: the portrait rail rides the classmates tab only
    // - the discovered faces refresh here (derive-on-read, zero
    // caching), the rail hides on the text-only tabs, and the text
    // body slides under the rail on the classmates tab.
    bool mates = _codexTab == 0;
    var codex = CampusCodex.Derive(_stage.EventEngine);
    var seats = CampusCodex.Seats();
    for (int i = 0; i &lt; _codexPortraits.Length; i++)
    {
        if (_codexPortraits[i] == null) continue;
        _codexPortraits[i].sprite = codex.Discovered(seats[i].key)
            ? CampusPlaceholder.ClassmateFace(seats[i].key)
            : CampusPlaceholder.LockedFace();
        _codexPortraits[i].gameObject.SetActive(mates);
    }
    for (int i = 0; i &lt; _codexFaculty.Length; i++)
        if (_codexFaculty[i] != null)
            _codexFaculty[i].gameObject.SetActive(mates);
    _albumLines.rectTransform.anchoredPosition = mates
```

```
        ? new Vector2(0, -770)
        : new Vector2(0, -660);
    _albumLines.rectTransform.sizeDelta = mates
        ? new Vector2(820, 120)
        : new Vector2(820, 340);
    RefreshCodexTabs();
}
/// &lt;summary&gt;Active tab reads full-bright with the accent label; the
/// idle tabs dim (a placeholder-era affordance, art wins in S2.5).&lt;/summary&gt;
void RefreshCodexTabs()
{
    for (int i = 0; i &lt; _codexTabs.Length; i++)
    {
        if (_codexTabs[i] == null) continue;
        bool active = i == _codexTab;
        _codexTabs[i].color = active ? Color.white : new Color(0.72f, 0.74f, 0.76f, 1f);
        _codexTabLabels[i].color = active
            ? CampusPlaceholder.AccentLime
            : new Color(0.62f, 0.64f, 0.66f, 1f);
    }
}
/// &lt;summary&gt;Refreshes the grade board plate off the shared BoardLine
/// data law (the view never formats grades itself).&lt;/summary&gt;
void RefreshBoard()
{
    if (_boardGrades != null)
        _boardGrades.text = CampusExamEngine.BoardLine(_stage.ExamEngine.History);
}
/// &lt;summary&gt;Freshest card fx off the engine log (cards carry the
/// base effect; chain beats stay pure telemetry).&lt;/summary&gt;
StatDelta FxOfCard(string id)
{
    var log = _stage.EventEngine.Log;
    for (int i = log.Count - 1; i &gt;= 0; i--)
        if (log[i].kind == CampusEventKind.Card &amp;&amp; log[i].id == id) return log[i].fx;
    return new StatDelta();
}
static string FxLine(StatDelta fx)
{
    var sb = new System.Text.StringBuilder();
    AppendFx(sb, CampusTexts.Text(CampusTexts.KeyStatIntell), fx.intell);
    AppendFx(sb, CampusTexts.Text(CampusTexts.KeyStatVit), fx.vit);
    AppendFx(sb, CampusTexts.Text(CampusTexts.KeyStatCharm), fx.charm);
    AppendFx(sb, CampusTexts.Text(CampusTexts.KeyStatMood), fx.mood);
    AppendFx(sb, CampusTexts.Text(CampusTexts.KeyStatPocket), fx.pocket);
    return sb.Length == 0 ? "-" : sb.ToString();
}
static void AppendFx(System.Text.StringBuilder sb, string label, int v)
{
    if (v == 0) return;
```

```
    if (sb.Length &gt; 0) sb.Append("  ");
    sb.Append(label).Append(v &gt; 0 ? " +" : " ").Append(v);
}
static string StatsLine(Stats5 s)
{
    return CampusTexts.Text(CampusTexts.KeyStatIntell) + " " + s.intell + "  "
        + CampusTexts.Text(CampusTexts.KeyStatVit) + " " + s.vit + "  "
        + CampusTexts.Text(CampusTexts.KeyStatCharm) + " " + s.charm + "\n"
        + CampusTexts.Text(CampusTexts.KeyStatMood) + " " + s.mood + "  "
        + CampusTexts.Text(CampusTexts.KeyStatPocket) + " " + s.pocket;
}
// U184 real-device CJK face (B569 chain step 3): resolve the UI font
// through the locale font set - the shipped name points at the B569
// NotoSansSC subset in Resources (real zh glyphs on device). Any miss
// (asset absent, name fallback) degrades to the built-in face, never
// a crash - the null-safe law.
static Font _uiFont;
static Font UiFont()
{
    if (_uiFont != null) return _uiFont;
    string name = "LegacyRuntime.ttf";
    if (ServiceLocator.TryGet&lt;ILocaleService&gt;(out var locale)) name = locale.GetFontSet().NormalFont;
    if (!string.IsNullOrEmpty(name) &amp;&amp; name != "LegacyRuntime.ttf")
    {
        int dot = name.LastIndexOf('.');
        var loaded = Resources.Load&lt;Font&gt;(dot &gt; 0 ? name.Substring(0, dot) : name);
        if (loaded != null) { _uiFont = loaded; return loaded; }
    }
    _uiFont = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
    return _uiFont;
}
Text MakeText(Transform parent, string name, string content, Vector2 anchored, int size, Color color)
{
    var go = new GameObject(name, typeof(Text));
    go.transform.SetParent(parent, false);
    var text = go.GetComponent&lt;Text&gt;();
    text.font = UiFont();
    text.fontSize = size;
    text.color = color;
    text.alignment = TextAnchor.MiddleCenter;
    text.horizontalOverflow = HorizontalWrapMode.Wrap;
    text.verticalOverflow = VerticalWrapMode.Overflow;
    var rt = text.rectTransform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 1f);
    rt.anchoredPosition = anchored;
    rt.sizeDelta = new Vector2(1000, size + 20);
    text.text = content;
    return text;
}
Image MakeBlock(Transform parent, string name, Sprite sprite, Vector2 anchored, float size)
```

```
        {
            var go = new GameObject(name, typeof(Image));
            go.transform.SetParent(parent, false);
            var image = go.GetComponent&lt;Image&gt;();
            image.sprite = sprite;
            image.SetNativeSize();
            var rt = image.rectTransform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 1f);
            rt.anchoredPosition = anchored;
            var scale = size / Mathf.Max(1f, rt.sizeDelta.x);
            rt.localScale = new Vector3(scale, scale, 1f);
            return image;
        }
        // Size a native block to an exact pixel size: divide by the ACTUAL
        // rect size after SetNativeSize (the pixelsPerUnit law turns a 64px
        // procedural sprite into a 100x100 rect - the old "/64" literals
        // rendered those panels 1.5625x too large, e.g. a 940px event card
        // frame painting 1469px wide, wider than the 1080 canvas; found by
        // DeviceMatrixCheck, G08-T025).
        static void ScaleTo(RectTransform rt, float w, float h)
        {
            rt.localScale = new Vector3(w / Mathf.Max(1f, rt.sizeDelta.x),
                h / Mathf.Max(1f, rt.sizeDelta.y), 1f);
        }
    }
}
// CampusTexts.cs
// CampusTexts.cs - the campus copy law for Crazy Campus (G08-T004).
// English source strings live here as compiled code (the family source law:
// code strings stay English); the zh-CN localization rides a separate data
// file (Assets/_Game/Resources/Texts_zhCN.json - data, not source; the
// Resources home keeps the table loadable in every channel build, the
// editor batch gates keep reading the same file by path). Lookup
// follows the shared LocaleService contract: a missing key falls back to the
// key itself (machine-readable, never a crash), and a zh entry missing from
// the data file falls back to the English source. Zero scattered copy: every
// UI-facing sentence routes through these keys - DataSchemaCheck scans the
// runtime tree for stray CJK and proves the coverage law.
using System;
using System.Collections.Generic;
using Sy.Core;
namespace Sy.CrazyCampus
{
    public static class CampusTexts
    {
        // ---- key constants (the compiled copy surface) ----
        public const string KeyHookCaption = "hook_caption";
        public const string KeyGenderBoy = "gender_boy";
        public const string KeyGenderGirl = "gender_girl";
        public const string KeyWeightStudy = "weight_study";
```

```
public const string KeyWeightPlay = "weight_play";
public const string KeyWeightClub = "weight_club";
public const string KeyWeightRest = "weight_rest";
public const string KeyStatIntell = "stat_intell";
public const string KeyStatVit = "stat_vit";
public const string KeyStatCharm = "stat_charm";
public const string KeyStatMood = "stat_mood";
public const string KeyStatPocket = "stat_pocket";
public const string KeyEventCardTitle = "event_card_title";
public const string KeyExamMidterm = "exam_midterm";
public const string KeyExamFinal = "exam_final";
public const string KeyExamResult = "exam_result";
public const string KeyExamPassed = "exam_passed";
public const string KeyExamFailed = "exam_failed";
// ---- exam show layer (T007 slice B): the guess-C comedy line, the
// repeat-year line, the grade-board frame and the short verdicts ----
public const string KeyExamGuessC = "exam_guess_c";
public const string KeyExamRepeat = "exam_repeat";
public const string KeyReportCard = "report_card";
public const string KeyExamPassShort = "exam_pass_short";
public const string KeyExamFailShort = "exam_fail_short";
public const string KeyMoodCrash = "mood_crash";
public const string KeyStageUp = "stage_up";
public const string KeyOfflineSummary = "offline_summary";
public const string KeyOfflineDouble = "offline_double";
public const string KeyAlbumTitle = "album_title";
public const string KeyAlbumShare = "album_share";
public const string KeyEndingScholar = "ending_scholar";
public const string KeyEndingArts = "ending_arts";
public const string KeyEndingSocial = "ending_social";
public const string KeyShareChallenge = "share_challenge";
public const string KeyDailyReport = "daily_report";
public const string KeyDayHud = "day_hud";
// ---- stage show layer (T008 slice B): the Crazy Channel MC banner,
// the repeat / graduate-segment lines, the ending matched / fallback
// lines, the carry label, the second-journey confirm flow and the
// empty-album line ----
public const string KeyStageUpMc = "stage_up_mc";
public const string KeyStageRepeatLine = "stage_repeat_line";
public const string KeyGraduateSegmentLine = "graduate_segment_line";
public const string KeyEndingMatched = "ending_matched";
public const string KeyEndingFallback = "ending_fallback";
public const string KeyCarryLabel = "carry_label";
public const string KeyNewJourney = "new_journey";
public const string KeyNewJourneyConfirm = "new_journey_confirm";
public const string KeyAlbumEmpty = "album_empty";
// ---- share face (T010): the card title, the context-less guard
// line and the saved-toast line (the album / ending share buttons) ----
public const string KeyShareCardTitle = "share_card_title";
public const string KeyShareNoContext = "share_no_context";
```

```
public const string KeyShareSaved = "share_saved";
// ---- codex tabs (T010 slice C): the three-book panel labels
// (classmates / events / graduation photos) ----
public const string KeyCodexTabClassmates = "codex_tab_classmates";
public const string KeyCodexTabEvents = "codex_tab_events";
public const string KeyCodexTabPhotos = "codex_tab_photos";
// ---- show-signal face (T023): the three hook-window show signals -
// first laugh (the first comedy card), crash flip (the two-way
// burnout), grad brag (the album long-image show-off moment) ----
public const string KeySignalFirstLaugh = "signal_first_laugh";
public const string KeySignalCrashFlip = "signal_crash_flip";
public const string KeySignalFlunkLine = "signal_flunk_line";
public const string KeySignalGradBrag = "signal_grad_brag";
// ---- stage display names (U096 round-1 fix): the ceremony face
// localizes the promoted stage name; the telemetry tokens (album
// entries, log details, check pins) keep the frozen English table ----
public const string KeyStageNamePrefix = "stage_name_";
// ---- event-card face (T006 slice B: frame, dismiss, the ten card
// stories, the five chain steps, the offline summary frame) ----
public const string KeyEventCardFrame = "event_card_frame";
public const string KeyEventCardDismiss = "event_card_dismiss";
public const string KeyEventE01 = "event_e01";
public const string KeyEventE02 = "event_e02";
public const string KeyEventE03 = "event_e03";
public const string KeyEventE04 = "event_e04";
public const string KeyEventE05 = "event_e05";
public const string KeyEventE06 = "event_e06";
public const string KeyEventE07 = "event_e07";
public const string KeyEventE08 = "event_e08";
public const string KeyEventE09 = "event_e09";
public const string KeyEventE10 = "event_e10";
public const string KeyChainDeskmateTruce = "chain_deskmate_truce";
public const string KeyChainAffinityUp = "chain_affinity_up";
public const string KeyChainConfiscated = "chain_confiscated";
public const string KeyChainClassReview = "chain_class_review";
public const string KeyChainTeacherMoved = "chain_teacher_moved";
public const string KeyOfflineSummaryFrame = "offline_summary_frame";
// ---- about face (T036): the AIGC-label duty keys - the explicit
// AI-content mark, the zero-RNG ten-pull disclosure, the daily ad
// caps, the age rating and the panel chrome ----
public const string KeyAboutButton = "about_button";
public const string KeyAboutTitle = "about_title";
public const string KeyAboutAigcLine = "about_aigc_line";
public const string KeyAboutOddsLine = "about_odds_line";
public const string KeyAboutAdcapLine = "about_adcap_line";
public const string KeyAboutAgeLine = "about_age_line";
public const string KeyAboutClose = "about_close";
/// &lt;summary&gt;English source table (compiled - the source-of-law face).&lt;/summary&gt;
public static readonly Dictionary&lt;string, string&gt; En = new Dictionary&lt;string, string&gt;
{
```

```
{ KeyHookCaption, "WELCOME TO CRAZY CAMPUS - YOUR SCHOOL LIFE STARTS NOW" },
{ KeyGenderBoy, "BOY" },
{ KeyGenderGirl, "GIRL" },
{ KeyWeightStudy, "STUDY" },
{ KeyWeightPlay, "PLAY" },
{ KeyWeightClub, "CLUB" },
{ KeyWeightRest, "REST" },
{ KeyStatIntell, "IQ" },
{ KeyStatVit, "VITALITY" },
{ KeyStatCharm, "CHARM" },
{ KeyStatMood, "MOOD" },
{ KeyStatPocket, "POCKET" },
{ KeyEventCardTitle, "CAMPUS EVENT" },
{ KeyExamMidterm, "MIDTERM" },
{ KeyExamFinal, "FINAL" },
{ KeyExamResult, "SCORE: {0}" },
{ KeyExamPassed, "PASSED!" },
{ KeyExamFailed, "OUCH... SEE YOU AT THE MAKEUP EXAM" },
{ KeyExamGuessC, "EVERY ANSWER IS C - PARTIAL CREDIT SAVED YOU!" },
{ KeyExamRepeat, "ONE MORE YEAR, SAME CLASSROOM..." },
{ KeyReportCard, "REPORT CARD" },
{ KeyExamPassShort, "PASS" },
{ KeyExamFailShort, "FAIL" },
{ KeyMoodCrash, "MOOD CRASH - TIME FOR A BREAK!" },
{ KeyStageUp, "STAGE UP! {0} UNLOCKED" },
{ KeyOfflineSummary, "WHILE YOU WERE GONE..." },
{ KeyOfflineDouble, "DOUBLE IT?" },
{ KeyAlbumTitle, "GROWTH ALBUM" },
{ KeyAlbumShare, "SHARE MY CAMPUS LIFE" },
{ KeyEndingScholar, "SCHOLAR ENDING" },
{ KeyEndingArts, "ALL-ROUNDER ENDING" },
{ KeyEndingSocial, "SOCIAL STAR ENDING" },
{ KeyShareChallenge, "CAN YOUR CAMPUS LIFE BEAT MINE?" },
{ KeyDailyReport, "DAY {0}: {1}" },
{ KeyDayHud, "DAY {0}" },
{ KeyEventCardFrame, "EVENT" },
{ KeyEventCardDismiss, "TAP TO DISMISS" },
{ KeyEventE01, "SNACK TIME TURNS INTO A WHOLE-CLASS FOOD PARTY." },
{ KeyEventE02, "NAP TIME ENDS WITH YOUR BLOCK TOWER ON THE FLOOR." },
{ KeyEventE03, "YOUR HOMEWORK DOODLE GOES VIRAL ON THE CLASS WALL." },
{ KeyEventE04, "A DESK-BORDER WAR WITH YOUR DESKMATE ENDS IN A TRUCE." },
{ KeyEventE05, "DODGEBALL PICKS YOU FIRST - EVERY SINGLE ROUND." },
{ KeyEventE06, "YOUR DRAWING WINS THE CLASS ART CORNER SPOTLIGHT." },
{ KeyEventE07, "THE SNACK CART SELLS OUT RIGHT BEFORE YOUR TURN." },
{ KeyEventE08, "YOUR HISTORY REPORT BECOMES THE CLASS COMEDY SKETCH." },
{ KeyEventE09, "YOU SHARE YOUR UMBRELLA - AND A FRIENDSHIP BUDS." },
{ KeyEventE10, "YOUR COMIC BOOK GETS CONFISCATED MID-CHAPTER." },
{ KeyChainDeskmateTruce, "TRUCE! THE DESK BORDER REOPENS." },
{ KeyChainAffinityUp, "GOODWILL +1 - YOUR DESKMATE NODS." },
{ KeyChainConfiscated, "THE TEACHER TAKES YOUR COMIC AWAY." },
```

```
        { KeyChainClassReview, "A CLASS REVIEW FOLLOWS - EVERYONE GIGGLES." },
        { KeyChainTeacherMoved, "THE TEACHER MOVES YOUR SEAT. COMEDY OVER." },
        { KeyOfflineSummaryFrame, "BACK AT SCHOOL" },
        { KeyStageUpMc, "CRAZY CHANNEL NEWS:" },
        { KeyStageRepeatLine, "GROWING-UP PENDING - ONE MORE YEAR, SAME CLASSROOM!" },
        { KeyGraduateSegmentLine, "FINAL YEAR: NO MORE EXAMS - ONLY THE ALBUM!" },
        { KeyEndingMatched, "THE REPORT FITS THE LINE - SIGNED!" },
        { KeyEndingFallback, "NO LINE FITS - THE STRONGEST STAT DECIDES!" },
        { KeyCarryLabel, "NEXT-KID CARRY: {0}" },
        { KeyNewJourney, "START A NEW JOURNEY" },
        { KeyNewJourneyConfirm, "TAP AGAIN - THE CARRY SEEDS YOUR NEXT KID" },
        { KeyAlbumEmpty, "NO MEMORIES YET - GO LIVE A SCHOOL LIFE!" },
        { KeyShareCardTitle, "MY CAMPUS LIFE" },
        { KeyShareNoContext, "GRADUATE FIRST - THE ALBUM NEEDS A FINALE!" },
        { KeyShareSaved, "CARD SAVED - SHOW IT OFF!" },
        { KeyCodexTabClassmates, "CLASSMATES" },
        { KeyCodexTabEvents, "EVENTS" },
        { KeyCodexTabPhotos, "GRAD PHOTOS" },
        { KeySignalFirstLaugh, "FIRST LAUGH!" },
        { KeySignalCrashFlip, "HOLY FLIP!" },
        { KeySignalFlunkLine, "FLUNK RISK - HIT THE BOOKS, KID!" },
        { KeySignalGradBrag, "GRAD BRAG - SHOW IT OFF!" },
        { KeyStageNamePrefix + "1", "KINDERGARTEN" },
        { KeyStageNamePrefix + "2", "PRIMARY" },
        { KeyStageNamePrefix + "3", "MIDDLE SCHOOL" },
        { KeyStageNamePrefix + "4", "HIGH SCHOOL" },
        { KeyStageNamePrefix + "5", "GRADUATE" },
        // about face (T036): the AIGC-label duty copy - the AI-content
        // mark, the zero-RNG ten-pull disclosure, the daily ad caps,
        // the age rating and the panel chrome
        { KeyAboutButton, "ABOUT" },
        { KeyAboutTitle, "ABOUT THIS GAME" },
        { KeyAboutAigcLine, "VISUAL ART ASSETS IN THIS GAME ARE AI-ASSISTED" },
        { KeyAboutOddsLine, "COSTUME TEN-PULL: {0}-SEAT POOL (OUTFITS {1} + TITLES {2}); FIXED-ORDER GRANTS, NO RARI
        { KeyAboutAdcapLine, "DAILY AD CAPS: EVENT SKIP {0}, OFFLINE DOUBLE {1}, SPEEDUP {2}, COSTUME PULL {3}; ALL
        { KeyAboutAgeLine, "AGE RATING: SUITABLE FOR AGES 8 AND UP" },
        { KeyAboutClose, "CLOSE" },
    };
    /// &lt;summary&gt;English lookup (missing key falls back to the key itself
    /// - machine-readable, never a crash).&lt;/summary&gt;
    public static string EnText(string key)
        =&gt; En.TryGetValue(key, out var v) &amp;&amp; !string.IsNullOrEmpty(v) ? v : key;
    /// &lt;summary&gt;Parses the zh-CN data file payload into a key-to-text
    /// map (values must be non-empty strings; anything else drops the
    /// entry so the lookup degrades to English, never to garbage).&lt;/summary&gt;
    public static Dictionary&lt;string, string&gt; ParseZh(string json)
    {
        var map = new Dictionary&lt;string, string&gt;();
        if (string.IsNullOrEmpty(json)) return map;
        try
```

```
            {
                if (!(MiniJson.Deserialize(json) is Dictionary&lt;string, object&gt; d)) return map;
                foreach (var kv in d)
                    if (!string.IsNullOrEmpty(kv.Key) &amp;&amp; kv.Value is string s &amp;&amp; !string.IsNullOrEmpty(s
                        map[kv.Key] = s;
            }
            catch
            {
                // corrupt data file = empty map = English fallback face
                return new Dictionary&lt;string, string&gt;();
            }
            return map;
        }
        /// &lt;summary&gt;LocaleService-compatible lookup delegate over a parsed
        /// zh map: zh-CN hits read the data file, everything else (and any
        /// miss) reads the English source. Wire-ready for the T005 UI face.&lt;/summary&gt;
        public static Func&lt;string, string, string&gt; LookupWithZh(Dictionary&lt;string, string&gt; zh)
        {
            return (lang, key) =&gt;
            {
                if (lang == "zh-CN" &amp;&amp; zh != null &amp;&amp; zh.TryGetValue(key, out var v) &amp;&amp; !string.I
                    return v;
                return EnText(key);
            };
        }
        /// &lt;summary&gt;Runtime zh face (U096 round-1 fix): the shipped game
        /// reads zh-CN first. The data file rides Resources so every channel
        /// build (editor / wechat / douyin / webgl) loads the same bytes -
        /// no file IO, which the mini-game channels do not have. Any miss,
        /// corrupt payload or absent asset degrades to the English source
        /// (the family degrade law, never a crash).&lt;/summary&gt;
        static Dictionary&lt;string, string&gt; _runtimeZh;
        public static string Text(string key)
        {
            if (_runtimeZh == null)
            {
                var asset = UnityEngine.Resources.Load&lt;UnityEngine.TextAsset&gt;("Texts_zhCN");
                _runtimeZh = asset != null ? ParseZh(asset.text) : new Dictionary&lt;string, string&gt;();
            }
            if (_runtimeZh.TryGetValue(key, out var v) &amp;&amp; !string.IsNullOrEmpty(v)) return v;
            return EnText(key);
        }
    }
}
// NoveltySchedule.cs
// NoveltySchedule.cs - authored staged-debut timetable for G08 (T033, U153 depth face).
// One row per school stage naming the content faces that debut (or step up)
// there - the "what is new this stage" teach face of the school career as
// data. NoveltyCheck asserts every row against the frozen event table, the
// codex cast map and the album booking faces, so the authored narrative can
```

```
// never drift from the shipped reality (the G04 T030 NoveltySchedule
// paradigm adapted to the campus career spine). Pure C#, deterministic,
// host-independent.
using System;
namespace Sy.CrazyCampus
{
    public sealed class StageNovelty
    {
        public int stage;          // 1..5 (kindergarten -&gt; graduate)
        public string[] newFaces;  // content faces debuting or stepping up here
        public string note;       // one-line teach intent
    }
    public static class NoveltySchedule
    {
        // Face vocabulary (validated by NoveltyCheck against the frozen tables):
        //   card_E01..E10       event card enters the stage-scoped pool
        //                      (stage 0 rows ride every pool, so they debut at stage 1)
        //   chain_truce         deskmate-war truce comedy family (E04)
        //   chain_teacher_moved three-step confiscation reversal family (E10)
        //   affinity_open       card-open affinity beat (E09)
        //   affinity_chain      chain-end affinity beat (E04)
        //   seat_&lt;key&gt;          codex cast seat becomes discoverable (mapped seats only;
        //                      swot + inventor ride undiscovered until their system faces wire in)
        //   album_ch2..ch5      promotion chapter face (the album books one entry on stage entry)
        //   album_ending        graduation ending face (terminal stage only)
        //   album_photo         graduation confession photo face (terminal stage only)
        public static readonly StageNovelty[] Rows =
        {
            new StageNovelty { stage = 1, newFaces = new string[] {
                "card_E01", "card_E02", "card_E07", "card_E09", "card_E10",
                "chain_teacher_moved", "affinity_open",
                "seat_foodie", "seat_sleeper", "seat_deskmate", "seat_gossip" },
                note = "kindergarten opens the pool, the any-stage trio, the reversal chain and four cast seats" },
            new StageNovelty { stage = 2, newFaces = new string[] {
                "card_E03", "card_E04", "card_E05", "card_E06",
                "chain_truce", "affinity_chain",
                "seat_lucky", "seat_athlete", "seat_artist", "album_ch2" },
                note = "primary debuts the truce chain, three cast seats and the first promotion chapter" },
            new StageNovelty { stage = 3, newFaces = new string[] {
                "card_E08", "seat_singer", "album_ch3" },
                note = "junior adds the singer seat and the third chapter" },
            new StageNovelty { stage = 4, newFaces = new string[] {
                "album_ch4" },
                note = "senior books the fourth chapter; the card pool rides the any-stage trio (senior card debt owned
            new StageNovelty { stage = 5, newFaces = new string[] {
                "album_ch5", "album_ending", "album_photo" },
                note = "graduation closes the album: the ending entry and the confession photo" },
        };
    }
}
```